

lr2xhgr

June 6, 2026

compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -
filename	lr2xhgr.dsp
maths.lib/author	GRAME
maths.lib/copyright	GRAME
maths.lib/license	LGPL with exception
maths.lib/name	Faust Math Library
maths.lib/version	2.9.0
name	lr2xhgr
routes.lib/name	Faust Signal Routing Library
routes.lib/version	1.3.0
seam.ambisonics.lib/author	Giuseppe Silvi
seam.ambisonics.lib/license	CC3
seam.ambisonics.lib/name	SEAM Ambisonics Library
seam.ambisonics.lib/version	0.1
seam.dwt.lib/author	Giuseppe Silvi
seam.dwt.lib/license	CC3
seam.dwt.lib/name	SEAM Discrete Wavelet Transform Library
seam.dwt.lib/version	0.1
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2
signals.lib/name	Faust Routing Library
signals.lib/version	1.6.0

This document provides a mathematical description of the Faust program text stored in the `lr2xhgr.dsp` file. See the notice in Section 3 (page 4) for details.

1 Mathematical definition of process

The *lr2xhgr* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$\begin{aligned} y_1(t) &= x_1(t) + x_2(t) \\ y_2(t) &= p_2(t) \cdot s_{15}(t) - p_{20}(t) \cdot s_{16}(t) \\ y_3(t) &= p_{21}(t) \cdot (s_9(t) + s_{12}(t)) + p_{14}(t) \cdot (s_{10}(t) + s_{13}(t)) \\ y_4(t) &= p_{20}(t) \cdot s_{15}(t) + p_2(t) \cdot s_{16}(t) \end{aligned}$$

2. Input signals x_i for $i \in [1, 2]$
3. User-interface input signals u_{s_i} for $i \in [1, 4]$ such that

"YAW Rotation around Z-axis"	$u_{s_1}(t) \in [-180, 180]$	(default value = 0)
"DIVERGENCE"	$u_{s_2}(t) \in [0, 360]$	(default value = 90)
"ROLL Rotation around X-axis"	$u_{s_3}(t) \in [-180, 180]$	(default value = 0)
"PITCH Rotation around Y-axis"	$u_{s_4}(t) \in [-180, 180]$	(default value = 0)

4. Intermediate signals p_i for $i \in [1, 21]$ and s_i for $i \in [1, 16]$ such that

$$\begin{aligned} p_1(t) &= 0.0174532925199433 \cdot u_{s_1}(t) \\ p_2(t) &= \cos(p_1(t)) \\ p_3(t) &= 0.00872664625997165 \cdot u_{s_2}(t) \\ p_4(t) &= \cos(p_3(t)) \\ p_5(t) &= 0.0174532925199433 \cdot u_{s_3}(t) \\ p_6(t) &= \cos(p_5(t)) \\ p_7(t) &= 0.5 \cdot p_6(t) \\ p_8(t) &= \sin(p_5(t)) \\ p_9(t) &= 0.25 \cdot p_8(t) \\ p_{10}(t) &= -1 \cdot p_3(t) \\ p_{11}(t) &= \cos(p_{10}(t)) \\ p_{12}(t) &= \sin(p_3(t)) \\ p_{13}(t) &= 0.0174532925199433 \cdot u_{s_4}(t) \\ p_{14}(t) &= \cos(p_{13}(t)) \\ p_{15}(t) &= 0.25 \cdot p_{14}(t) \\ p_{16}(t) &= \sin(p_{13}(t)) \\ p_{17}(t) &= 0.5 \cdot p_8(t) \\ p_{18}(t) &= 0.25 \cdot p_6(t) \\ p_{19}(t) &= \sin(p_{10}(t)) \\ p_{20}(t) &= \sin(p_1(t)) \\ p_{21}(t) &= 0.25 \cdot p_{16}(t) \end{aligned}$$

$$\begin{aligned}
s_1(t) &= (x_1(t) - x_1(t-1)) \\
s_2(t) &= x_1(t) + x_1(t-1) \\
s_3(t) &= (s_2(t) - s_2(t-1)) \\
s_4(t) &= (p_7(t) \cdot s_1(t) - p_9(t) \cdot s_3(t)) \\
s_5(t) &= (x_2(t) - x_2(t-1)) \\
s_6(t) &= x_2(t) + x_2(t-1) \\
s_7(t) &= (s_6(t) - s_6(t-1)) \\
s_8(t) &= (p_7(t) \cdot s_5(t) - p_9(t) \cdot s_7(t)) \\
s_9(t) &= (s_2(t) + s_2(t-1)) \\
s_{10}(t) &= (p_{17}(t) \cdot s_1(t) + p_{18}(t) \cdot s_3(t)) \\
s_{11}(t) &= (p_{15}(t) \cdot s_9(t) - p_{16}(t) \cdot s_{10}(t)) \\
s_{12}(t) &= (s_6(t) + s_6(t-1)) \\
s_{13}(t) &= (p_{17}(t) \cdot s_5(t) + p_{18}(t) \cdot s_7(t)) \\
s_{14}(t) &= (p_{15}(t) \cdot s_{12}(t) - p_{16}(t) \cdot s_{13}(t)) \\
s_{15}(t) &= (p_4(t) \cdot s_4(t) + p_{11}(t) \cdot s_8(t) - p_{12}(t) \cdot s_{11}(t) + p_{19}(t) \cdot s_{14}(t)) \\
s_{16}(t) &= (p_{11}(t) \cdot s_{14}(t) + p_{12}(t) \cdot s_4(t) + p_4(t) \cdot s_{11}(t) + p_{19}(t) \cdot s_8(t))
\end{aligned}$$

2 Block diagram of process

The block diagram of **process** is shown on Figure 1 (page 3).

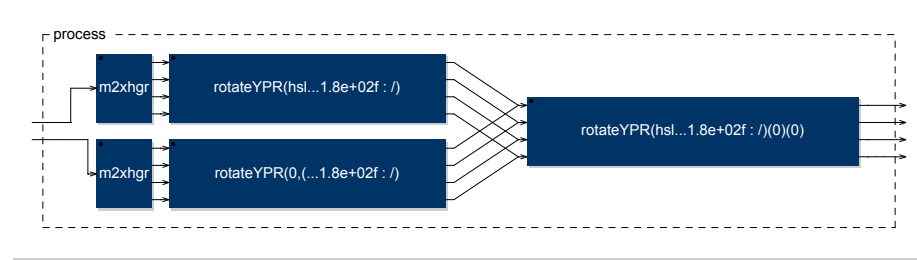


Figure 1: Block diagram of **process**

3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_s sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `lr2xhgr-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the L^AT_EX source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `lr2xhgr.dsp`

```
1 import("seam.lib");
2 // Canonical DSP: sam.lr2xhgr (Haar via seam.dwt.lib) in seam.ambisonics.lib
```

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

```

3 divergence = hslider("[00]DIVERGENCE",90,0,360,0.1) / 2 : ma.deg2rad;
4 yaw = hslider("[01]YAW Rotation around Z-axis",0,-180,180,0.5) : ma.deg2rad;
5 pitch = hslider("[02]PITCH Rotation around Y-axis",0,-180,180,0.5) : ma.deg2rad;
6 roll = hslider("[03]ROLL Rotation around X-axis",0,-180,180,0.5) : ma.deg2rad;
7 process = sam.lr2xhgr(divergence, yaw, pitch, roll);

```

Listing 2: seam.lib

```

1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
9 san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 // ### author specific literature
25 // sds = library("seam.discipio.lib");
26 smg = library("seam.gerzon.lib");
27 // sln = library("seam.nono.lib");
28 // sgn = library("seam.nottoli.lib");
29 sjm = library("seam.moorer.lib");
30 scr = library("seam.roads.lib");
31 sms = library("seam.schroeder.lib");
32 sfv = library("seam.freeverb.lib");
33
34 sff = library("seam.ffunctions.lib");
35
36 // ### live electronics stuff
37 // sgu = library("seam.gui.lib");
38 // shw = library("seam.hardware.lib");
39 // smx = library("seam.mixer.lib");
40
41 // ### instrument specific literature
42 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1 //##### stdfaust.lib #####
2 // The purpose of this library is to give access to all the Faust standard libraries
3 // through a series of environments.
4 //#####
5
6 aa = library("aanl.lib");
7 sf = library("all.lib");

```

```

8  an = library("analyzers.lib");
9  ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.ambisonics.lib

```

1  declare name "SEAM Ambisonics Library";
2  declare version "0.1";
3  declare author "Giuseppe Silvi";
4  declare license "CC3";
5  //
6  // SEAM Ambisonics Library
7  //
8  // Functions for first-order Ambisonics processing in ACN/SN3D (AmbiX)
9  // format, including format conversion from B-format (FuMa) and
10 // 3D rotation.
11 //
12 // Channel ordering follows the ACN convention:
13 // a0 = W (omnidirectional)
14 // a1 = Y (front-back, left-right)
15 // a2 = Z (up-down)
16 // a3 = X (front-back)
17 //
18 // References:
19 // - Furuse-Malham, "A 3-D Sound Primer" (1998)
20 // - Nachbar et al., "ambiX -- A Suggested Ambisonics Format" (2011)
21 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
22 // - Malham, "3-D Sound Spatialization using Ambisonic Techniques" (1999)
23 //
24 //
25 //
26 import("seam.lib");

```

```

27 //
28 sam = library("seam.ambisonics.lib");
29 //
30 //           B-format to AmbiX conversion
31 //
32 // Converts a first-order Ambisonics signal from B-format (FuMa)
33 // to AmbiX (ACN/SN3D).
34 //
35 // B-format (FuMa): W, X, Y, Z -- with W attenuated by 1/sqrt(2)
36 // AmbiX (ACN/SN3D): W, Y, Z, X -- with W at unity gain
37 //
38 // The conversion performs two operations:
39 // 1. Rescale W by sqrt(2) to restore unity gain
40 // 2. Reorder channels from FuMa (WXYZ) to ACN (WYZX)
41 //
42 btow(w,x,y,z) = a0, a1, a2, a3
43 with {
44     a0 = w * sqrt(2); // W: restore unity gain
45     a1 = y;           // Y: FuMa -> ACN reorder
46     a2 = z;           // Z: unchanged
47     a3 = x;           // X: FuMa -> ACN reorder
48 };
49 //
50 //           AmbiX -> tetrahedral decoder
51 //
52 // Decodes a first-order AmbiX signal to four loudspeakers placed at
53 // the vertices of a tetrahedron inscribed in a cube:
54 //
55 // LFU -- Left Front Up    ( +Y +Z +X )
56 // RFD -- Right Front Down ( -Y -Z +X )
57 // RBU -- Right Back Up    ( -Y +Z -X )
58 // LBD -- Left Back Down   ( +Y -Z -X )
59 //
60 // Each output is the sum of W and the three Cartesian components
61 // signed according to the corner's position, scaled by 1/2:
62 //
63 // LFU = (a0 + a1 + a2 + a3) / 2
64 // RFD = (a0 - a1 - a2 + a3) / 2
65 // RBU = (a0 - a1 + a2 - a3) / 2
66 // LBD = (a0 + a1 - a2 - a3) / 2
67 //
68 // The matrix is the AmbiX-domain analogue of Gerzon's `bamodule`
69 // (FuMa B-format -> tetrahedron) found in seam.gerzon.lib. The two
70 // differ only in the input reference system; the same diagonal-cube
71 // vertex layout is preserved.
72 //
73 // Compensation shelving filters (Gerzon 1975) are deliberately
74 // omitted: the STONE tetrahedral loudspeaker amplifier handles
75 // frequency-domain correction downstream.
76 //
77 // References:
78 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
79 // - Malham, "Space in Music -- Music in Space" (1998)
80 //
81 bamodulex(a0,a1,a2,a3) = lfu, rfd, rbu, lbd
82 with {
83     lfu = (a0 + a1 + a2 + a3) / 2;
84     rfd = (a0 - a1 - a2 + a3) / 2;
85     rbu = (a0 - a1 + a2 - a3) / 2;
86     lbd = (a0 + a1 - a2 - a3) / 2;
87 };
88 //
89 //           First-order AmbiX rotation
90 //
91 // Applies 3D rotation to a first-order AmbiX signal.
92 // Rotation is applied in intrinsic order: Roll -> Pitch -> Yaw.
93 // The resulting matrix is R = Rz(yaw) * Ry(pitch) * Rx(roll).
94 //

```

```

95 // Roll (X-axis) -- lateral tilt (left-right vertical plane)
96 // Pitch (Y-axis) -- forward-backward tilt (vertical plane)
97 // Yaw (Z-axis) -- horizontal rotation (azimuth)
98 //
99 // Channel W (omnidirectional) is unaffected by rotation.
100 //
101 // Arguments:
102 // yaw, pitch, roll -- rotation angles in radians
103 // a0, a1, a2, a3 -- input channels in ACN order (W, Y, Z, X)
104 //
105 rotateYPR(yaw, pitch, roll, a0, a1, a2, a3) =
106     a0, a1_ypr, a2_ypr, a3_ypr
107 with {
108     // Roll (X-axis): rotates Y and Z
109     a1_r = a1*cos(roll) - a2*sin(roll);
110     a2_r = a1*sin(roll) + a2*cos(roll);
111     a3_r = a3;
112     // Pitch (Y-axis): rotates Z and X
113     a1_rp = a1_r;
114     a2_rp = a2_r*cos(pitch) + a3_r*sin(pitch);
115     a3_rp = -a2_r*sin(pitch) + a3_r*cos(pitch);
116     // Yaw (Z-axis): rotates Y and X
117     a1_ypr = a1_rp*cos(yaw) - a3_rp*sin(yaw);
118     a2_ypr = a2_rp;
119     a3_ypr = a1_rp*sin(yaw) + a3_rp*cos(yaw);
120 };
121 //
122 //----- UHJ C-FORMAT ----
123 // AmbiX (ACN/SN3D) -> UHJ C-format L,R,T,Q.
124 // Gerzon, "Ambisonics in Multichannel Broadcasting and Video", JAES 1983.
125 // Quadrature all-pass pair re-derived analytically (SEAM-LTM x2uhj, 2026);
126 // max phase error 1.36 deg over 20 Hz-20 kHz. Coefficients are the single
127 // source of truth in seam-ltm plugins/x2uhj/source/x2uhj_coeffs.h.
128 // NB canonical structure: every component passes the real network hR; only the
129 // j-terms use the imaginary network hI. The 90 deg is the DIFFERENCE between
130 // the two matched networks. Earlier drafts leaving the real path unfiltered
131 // are superseded.
132 //
133 // 2nd-order RBJ all-pass parameterised by (frequency, Q):
134 apuhj(f,Q) = fi.tf2(a2, a1, 1.0, a1, a2)
135 with {
136     w = 2.0*ma.PI*f/ma.SR;
137     alpha = sin(w)/(2.0*Q);
138     n = 1.0 + alpha;
139     a1 = -2.0*cos(w)/n;
140     a2 = (1.0 - alpha)/n;
141 };
142 // Matched quadrature pair (3 sections each):
143 hRuhj = apuhj(141.887012,0.201947) : apuhj(671.745773,0.212154) : apuhj
(18654.238494,0.303097);
144 hIuhj = apuhj(24.003923,0.309038) : apuhj(2991.937233,0.384766) : apuhj
(3219.986192,0.096342);
145 // AmbiX ACN inputs in order (W,Y,Z,X) -> UHJ (L,R,T,Q):
146 x2uhj(aW,aY,aZ,aX) = L,R,T,Q
147 with {
148     W = aW/sqrt(2.0); X = aX; Y = aY; Z = aZ; // ACN/SN3D -> FuMa
149     Wr = hRuhj(W); Xr = hRuhj(X); Yr = hRuhj(Y); Zr = hRuhj(Z);
150     Wi = hIuhj(W); Xi = hIuhj(X);
151     S = 0.9396926*Wr + 0.1855740*Xr;
152     D = (-0.3420201*Wi + 0.5098604*Xi) + 0.6554516*Yr;
153     L = 0.5*(S + D);
154     R = 0.5*(S - D);
155     T = (-0.1432*Wi + 0.6512*Xi) - 0.7071*Yr;
156     Q = 0.9772*Zr;
157 };
158 //process = x2uhj;
159 //
160 //----- MONO -> AMBIX (HAAR) ----

```



```

161 // Mono to first-order AmbiX via the Haar wavelet decomposition
162 // (sdw.haarmn from seam.dwt.lib). Channel 0 (W) is the passthrough; the
163 // QMF sub-bands form the directional components. Rotation, if any, is
164 // applied separately with rotateYPR. SEAM-LTM M2XHGR plugin.
165 m2xhgr = sdw.haarmn(1);
166 //process = no.noise : m2xhgr;
167 //
168 //----- STEREO -> AMBIX (HAAR) ---
169 // Stereo (L,R) to first-order AmbiX via Haar, with a divergence control.
170 // Each channel is Haar-encoded (m2xhgr), rotated by +/-divergence, summed,
171 // then globally rotated by yaw. divergence/yaw/pitch/roll in radians.
172 // SEAM-LTM LR2XHGR plugin.
173 lr2xhgr(divergence, yaw, pitch, roll) =
174     par(i, 2, m2xhgr) :
175     rotateYPR(divergence, pitch, roll),
176     rotateYPR(0 - divergence, pitch, roll)
177     :-> si.bus(4) :
178     rotateYPR(yaw, 0, 0);
179 //process = (no.noise, no.noise) : lr2xhgr(ma.PI/2, 0, 0, 0);

```

Listing 5: maths.lib

```

1 //##### maths.lib #####
2 // Maths library. Its official prefix is `ma`.
3 //
4 // This library provides mathematical functions and utilities for numerical
5 // computations in Faust. It includes trigonometric, exponential, logarithmic, and
6 // statistical functions, constants, and complex-number
7 // operations used throughout Faust DSP and control code.
8 //
9 // The Maths library is organized into 1 section:
10 //
11 // * [Functions Reference](#functions-reference)
12 //
13 // Some functions are implemented as Faust foreign functions of `math.h` functions
14 // that are not part of Faust's primitives. Defines also various constants and several
15 // utilities.
16 //
17 // #### References
18 //
19 // <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
20 //#####
21
22 // ## History
23 // * 06/13/2016 [RM] normalizing and integrating to new libraries
24 // * 07/08/2015 [Y0] documentation comments
25 // * 20/06/2014 [SL] added FTZ function
26 // * 22/06/2013 [Y0] added float|double|quad variants of some foreign functions
27 // * 28/06/2005 [Y0] postfix functions with 'f' to force float version instead of double
28 // * 28/06/2005 [Y0] removed 'modf' because it requires a pointer as argument
29
30 /*****
31 *****/
32 FAUST library file
33 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
34 -----
35 This program is free software; you can redistribute it and/or modify
36 it under the terms of the GNU Lesser General Public License as
37 published by the Free Software Foundation; either version 2.1 of the
38 License, or (at your option) any later version.
39
40 This program is distributed in the hope that it will be useful,
41 but WITHOUT ANY WARRANTY; without even the implied warranty of
42 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

```

```

43 | GNU Lesser General Public License for more details.
44 |
45 | You should have received a copy of the GNU Lesser General Public
46 | License along with the GNU C Library; if not, write to the Free
47 | Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
48 | 02111-1307 USA.
49 |
50 | EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
51 | larger FAUST program which directly or indirectly imports this library
52 | file and still distribute the compiled code generated by the FAUST
53 | compiler, or a modified version of this compiled code, under your own
54 | copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
55 | grants you the right to freely choose the license for the resulting
56 | compiled code. In particular the resulting compiled code has no obligation
57 | to be LGPL or GPL. For example you are free to choose a commercial or
58 | closed source license or any other license if you decide so.
59 | *****
60 | *****/
61 |
62 | // This library contains platform specific constants
63 | ba = library("basics.lib");
64 | pl = library("platform.lib");
65 | ma = library("maths.lib"); // for compatible copy/paste out of this file
66 |
67 | declare name "Faust Math Library";
68 | declare version "2.9.0";
69 | declare author "GRAMME";
70 | declare copyright "GRAMME";
71 | declare license "LGPL with exception";
72 |
73 | //=====Functions Reference=====
74 | //=====
75 |
76 |
77 | //-----`(ma.)SR`-----
78 | // Current sampling rate given at init time. Constant during program execution.
79 | //
80 | // #### Usage
81 | //
82 | // ```
83 | // SR : _
84 | // ```
85 | //
86 | // Where:
87 | //
88 | // * `SR`: initialization-time sampling rate constant
89 | //
90 | // #### Test
91 | // ```
92 | // ma = library("maths.lib");
93 | // SR_test = ma.SR;
94 | // ```
95 | //-----
96 | SR = pl.SR;
97 |
98 | //-----`(ma.)T`-----
99 | // Current sample duration in seconds computed from the sampling rate given at init time.
   Constant during program execution.
100 | //
101 | // #### Usage
102 | //
103 | // ```
104 | // T : _
105 | // ```
106 | //
107 | // Where:
108 | //
109 | // * `T`: sample duration ( $1/SR$ ) constant

```

```

110 //
111 // #### Test
112 // ```
113 // ma = library("maths.lib");
114 // T_test = ma.T;
115 // ```
116 //-----
117 T = 1.0 / SR;
118
119 //-----`(ma.)BS`-----
120 // Current block-size. Can change during the execution at each block.
121 //
122 // #### Usage
123 //
124 // ```
125 // BS : _
126 // ```
127 //
128 // Where:
129 //
130 // * `BS`: current processing block size
131 //
132 // #### Test
133 // ```
134 // ma = library("maths.lib");
135 // BS_test = ma.BS;
136 // ```
137 //-----
138 BS = pl.BS;
139
140
141 //-----`(ma.)PI`-----
142 // Constant PI in double precision.
143 //
144 // #### Usage
145 //
146 // ```
147 // PI : _
148 // ```
149 //
150 // Where:
151 //
152 // * `PI`: double-precision constant
153 //
154 // #### Test
155 // ```
156 // ma = library("maths.lib");
157 // PI_test = ma.PI;
158 // ```
159 //-----
160 PI = 3.14159265358979323846;
161
162
163 //-----`(ma.)deg2rad`-----
164 // Convert degrees to radians.
165 //
166 // #### Usage
167 //
168 // ```
169 // 45. : deg2rad
170 // ```
171 //
172 // Where:
173 //
174 // * input: angle in degrees to convert
175 //
176 // #### Test
177 // ```

```

```

178 // ma = library("maths.lib");
179 // deg2rad_test = 45.0 : ma.deg2rad;
180 // ```
181 //-----
182 deg2rad = _ * PI / 180.;
183
184
185 //-----`(ma.)rad2deg`-----
186 // Convert radians to degrees.
187 //
188 // #### Usage
189 //
190 // ```
191 // ma.PI : rad2deg
192 // ```
193 //
194 // Where:
195 //
196 // * input: angle in radians to convert
197 //
198 // #### Test
199 // ```
200 // ma = library("maths.lib");
201 // rad2deg_test = ma.PI : ma.rad2deg;
202 // ```
203 //-----
204 rad2deg = _ * 180. / PI;
205
206
207 //-----`(ma.)E`-----
208 // Constant e in double precision.
209 //
210 // #### Usage
211 //
212 // ```
213 // E : _
214 // ```
215 //
216 // Where:
217 //
218 // * `E`: double-precision Euler's number constant
219 //
220 // #### Test
221 // ```
222 // ma = library("maths.lib");
223 // E_test = ma.E;
224 // ```
225 //-----
226 E = 2.71828182845904523536;
227
228
229 //-----`(ma.)EPSILON`-----
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
232 // and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
233 // that is smallest positive number such that `1.0 + EPSILON != 1.0`.
234 //
235 // #### Usage
236 //
237 // ```
238 // EPSILON : _
239 // ```
240 //
241 // Where:
242 //
243 // * `EPSILON`: machine epsilon constant for the current floating-point precision
244 //
245 // #### Test

```

```

246 //  ```
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 //  ```
250 //-----
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.084202172485504434007452e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255
256
257 //-----`(ma.)MIN`-----
258 // Constant MIN available in simple/double/quad precision (minimal positive value).
259 //
260 // #### Usage
261 //
262 //  ```
263 // MIN : _
264 //  ```
265 //
266 // Where:
267 //
268 // * `MIN`: minimal positive normalized value for the current precision
269 //
270 // #### Test
271 //  ```
272 // ma = library("maths.lib");
273 // MIN_test = ma.MIN * 1e307;
274 //  ```
275 //-----
276 singleprecision MIN = 1.175494351e-38;
277 doubleprecision MIN = 2.2250738585072014e-308;
278 quadprecision MIN = 2.2250738585072014e-308;
279 fixedpointprecision MIN = 2.2250738585072014e-308;
280
281
282 //-----`(ma.)MAX`-----
283 // Constant MAX available in simple/double/quad precision (maximal positive value).
284 //
285 // #### Usage
286 //
287 //  ```
288 // MAX : _
289 //  ```
290 //
291 // Where:
292 //
293 // * `MAX`: maximal finite value for the current precision
294 //
295 // #### Test
296 //  ```
297 // ma = library("maths.lib");
298 // MAX_test = ma.MAX;
299 //  ```
300 //-----
301 singleprecision MAX = 3.402823466e+38;
302 doubleprecision MAX = 1.7976931348623158e+308;
303 quadprecision MAX = 1.7976931348623158e+308;
304 fixedpointprecision MAX = 1.7976931348623158e+308;
305
306 // Obsolete, kept for compatibility reasons
307 INFINITY = MAX;
308
309 //-----`(ma.)FTZ`-----
310 // Flush to zero: force samples under the "maximum subnormal number"
311 // to be zero. Usually not needed in C++ because the architecture
312 // file take care of this, but can be useful in JavaScript for instance.
313 //

```

```

314 // #### Usage
315 //
316 // ```
317 // _ : FTZ : _
318 // ```
319 //
320 // Where:
321 //
322 // * `x`: input signal to flush if its magnitude is subnormal
323 //
324 // #### Test
325 // ```
326 // ma = library("maths.lib");
327 // FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
328 // ```
329 //
330 // #### References
331 //
332 // * <http://docs.oracle.com/cd/E19957-01/806-3568/ncg\_math.html>
333 //-----
334 FTZ(x) = x * (abs(x) > MIN);
335
336
337 //-----`(ma.)copysign`-----
338 // Changes the sign of x (first input) to that of y (second input).
339 //
340 // #### Usage
341 //
342 // ```
343 // _ : copysign : _
344 // ```
345 //
346 // Where:
347 //
348 // * `x`: value whose magnitude is preserved
349 // * `y`: value providing the sign
350 //
351 // #### Test
352 // ```
353 // ma = library("maths.lib");
354 // copysign_test = (-1.0, 2.0) : ma.copysign;
355 // ```
356 //-----
357 copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");
358
359
360 //-----`(ma.)neg`-----
361 // Invert the sign (-x) of a signal.
362 //
363 // #### Usage
364 //
365 // ```
366 // _ : neg : _
367 // ```
368 //
369 // Where:
370 //
371 // * `x`: value to negate
372 //
373 // #### Test
374 // ```
375 // ma = library("maths.lib");
376 // neg_test = 3.5 : ma.neg;
377 // ```
378 //-----
379 neg(x) = -x;
380
381

```

```

382 //-----`(ma.)not`-----
383 // Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-
    primitive) as `not(x) = x xor -1;`.
384 // So working regardless of the size of the integer, assuming negative numbers in two's
    complement.
385 //
386 // #### Usage
387 //
388 // ```
389 // _ : not : _
390 // ```
391 //
392 // Where:
393 //
394 // * `x`: integer input value
395 //
396 // #### Test
397 // ```
398 // ma = library("maths.lib");
399 // not_test = 5 : ma.not;
400 // ```
401 //-----
402 not(x) = x xor -1;
403
404
405 //-----`(ma.)sub(x,y)`-----
406 // Subtract `x` and `y`.
407 //
408 // #### Usage
409 //
410 // ```
411 // _ : sub : _
412 // ```
413 //
414 // Where:
415 //
416 // * `x`: first operand
417 // * `y`: second operand
418 //
419 // #### Test
420 // ```
421 // ma = library("maths.lib");
422 // sub_test = (3, 10) : ma.sub;
423 // ```
424 //-----
425 sub(x,y) = y-x;
426
427
428 //-----`(ma.)inv`-----
429 // Compute the inverse (1/x) of the input signal.
430 //
431 // #### Usage
432 //
433 // ```
434 // _ : inv : _
435 // ```
436 //
437 // Where:
438 //
439 // * `x`: denominator input (non-zero)
440 //
441 // #### Test
442 // ```
443 // ma = library("maths.lib");
444 // inv_test = 4.0 : ma.inv;
445 // ```
446 //-----
447 inv(x) = 1/x;

```

```

448
449
450 //-----`(ma.)cbrt`-----
451 // Computes the cube root of of the input signal.
452 //
453 // #### Usage
454 //
455 // ```
456 // _ : cbrt : _
457 // ```
458 //
459 // Where:
460 //
461 // * `x`: value whose cube root is computed
462 //
463 // #### Test
464 //
465 // ma = library("maths.lib");
466 // cbrt_test = 8.0 : ma.cbrt;
467 //
468 //-----
469 cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");
470
471
472 //-----`(ma.)hypot`-----
473 // Computes the euclidian distance of the two input signals
474 // sqrt(x*x+y*y) without undue overflow or underflow.
475 //
476 // #### Usage
477 //
478 // ```
479 // _ : hypot : _
480 // ```
481 //
482 // Where:
483 //
484 // * `x`: first operand
485 // * `y`: second operand
486 //
487 // #### Test
488 //
489 // ma = library("maths.lib");
490 // hypot_test = (3.0, 4.0) : ma.hypot;
491 //
492 //-----
493 hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");
494
495
496 //-----`(ma.)ldexp`-----
497 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
498 //
499 // #### Usage
500 //
501 // ```
502 // _ : ldexp : _
503 // ```
504 //
505 // Where:
506 //
507 // * `x`: significand input
508 // * `n`: exponent (integer) input
509 //
510 // #### Test
511 //
512 // ma = library("maths.lib");
513 // ldexp_test = (1.5, 3) : ma.ldexp;
514 //
515 //-----

```



```

516 ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");
517
518
519 //-----`(ma.)scalb`-----
520 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
521 //
522 // #### Usage
523 //
524 // ```
525 // _ : scalb : _
526 // ```
527 //
528 // Where:
529 //
530 // * `x`: significand input
531 // * `n`: exponent (integer) input
532 //
533 // #### Test
534 // ```
535 // ma = library("maths.lib");
536 // scalb_test = (2.0, -1) : ma.scalb;
537 // ```
538 //-----
539 scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");
540
541
542 //-----`(ma.)log1p`-----
543 // Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
544 //
545 // #### Usage
546 //
547 // ```
548 // _ : log1p : _
549 // ```
550 //
551 // Where:
552 //
553 // * `x`: offset used in `log(1 + x)` (must be greater than -1)
554 //
555 // #### Test
556 // ```
557 // ma = library("maths.lib");
558 // log1p_test = 0.5 : ma.log1p;
559 // ```
560 //-----
561 log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");
562
563
564 //-----`(ma.)logb`-----
565 // Return exponent of the input signal as a floating-point number.
566 //
567 // #### Usage
568 //
569 // ```
570 // _ : logb : _
571 // ```
572 //
573 // Where:
574 //
575 // * `x`: positive value whose exponent part is returned
576 //
577 // #### Test
578 // ```
579 // ma = library("maths.lib");
580 // logb_test = 8.0 : ma.logb;
581 // ```
582 //-----
583 logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");

```

```

584
585
586 //-----`(ma.)ilogb`-----
587 // Return exponent of the input signal as an integer number.
588 //
589 // #### Usage
590 //
591 // ```
592 // _ : ilogb : _
593 // ```
594 //
595 // Where:
596 //
597 // * `x`: positive value whose exponent part is returned
598 //
599 // #### Test
600 //
601 // ma = library("maths.lib");
602 // ilogb_test = 8.0 : ma.ilogb;
603 //
604 //-----
605 ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");
606
607
608 //-----`(ma.)log2`-----
609 // Returns the base 2 logarithm of x.
610 //
611 // #### Usage
612 //
613 // ```
614 // _ : log2 : _
615 // ```
616 //
617 // Where:
618 //
619 // * `x`: positive value whose base-2 logarithm is computed
620 //
621 // #### Test
622 //
623 // ma = library("maths.lib");
624 // log2_test = 8.0 : ma.log2;
625 //
626 //-----
627 log2(x) = log(x)/log(2.0);
628
629
630 //-----`(ma.)expm1`-----
631 // Return exponent of the input signal minus 1 with better precision.
632 //
633 // #### Usage
634 //
635 // ```
636 // _ : expm1 : _
637 // ```
638 //
639 // Where:
640 //
641 // * `x`: input value used for the `exp(x) - 1` computation
642 //
643 // #### Test
644 //
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 //
648 //-----
649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651

```

```

652 //-----`(ma.)acosh`-----
653 // Computes the principle value of the inverse hyperbolic cosine
654 // of the input signal.
655 //
656 // #### Usage
657 //
658 // ```
659 // _ : acosh : _
660 // ```
661 //
662 // Where:
663 //
664 // * `x`: input value (greater than or equal to 1)
665 //
666 // #### Test
667 // ```
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // ```
671 //-----
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674
675 //-----`(ma.)asinh`-----
676 // Computes the inverse hyperbolic sine of the input signal.
677 //
678 // #### Usage
679 //
680 // ```
681 // _ : asinh : _
682 // ```
683 //
684 // Where:
685 //
686 // * `x`: input value
687 //
688 // #### Test
689 // ```
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // ```
693 //-----
694 asinh = ffunction(float asinhf|asinh|asinh1 (float), <math.h>, "");
695
696
697 //-----`(ma.)atanh`-----
698 // Computes the inverse hyperbolic tangent of the input signal.
699 //
700 // #### Usage
701 //
702 // ```
703 // _ : atanh : _
704 // ```
705 //
706 // Where:
707 //
708 // * `x`: input value in (-1, 1)
709 //
710 // #### Test
711 // ```
712 // ma = library("maths.lib");
713 // atanh_test = 0.5 : ma.atanh;
714 // ```
715 //-----
716 atanh = ffunction(float atanhf|atanh|atanhl (float), <math.h>, "");
717
718
719 //-----`(ma.)sinh`-----

```

```

720 // Computes the hyperbolic sine of the input signal.
721 //
722 // #### Usage
723 //
724 // ```
725 // _ : sinh : _
726 // ```
727 //
728 // Where:
729 //
730 // * `x`: input value
731 //
732 // #### Test
733 //
734 // ma = library("maths.lib");
735 // sinh_test = 0.5 : ma.sinh;
736 // ```
737 //-----
738 sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");
739
740
741 //-----`(ma.)cosh`-----
742 // Computes the hyperbolic cosine of the input signal.
743 //
744 // #### Usage
745 //
746 // ```
747 // _ : cosh : _
748 // ```
749 //
750 // Where:
751 //
752 // * `x`: input value
753 //
754 // #### Test
755 //
756 // ma = library("maths.lib");
757 // cosh_test = 0.5 : ma.cosh;
758 // ```
759 //-----
760 cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");
761
762
763 //-----`(ma.)tanh`-----
764 // Computes the hyperbolic tangent of the input signal.
765 //
766 // #### Usage
767 //
768 // ```
769 // _ : tanh : _
770 // ```
771 //
772 // Where:
773 //
774 // * `x`: input value
775 //
776 // #### Test
777 //
778 // ma = library("maths.lib");
779 // tanh_test = 0.5 : ma.tanh;
780 // ```
781 //-----
782 tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");
783
784
785 //-----`(ma.)erf`-----
786 // Computes the error function of the input signal.
787 //

```

```

788 // #### Usage
789 //
790 // ```
791 // _ : erf : _
792 // ```
793 //
794 // Where:
795 //
796 // * `x`: input value
797 //
798 // #### Test
799 // ```
800 // ma = library("maths.lib");
801 // erf_test = 0.5 : ma.erf;
802 // ```
803 //-----
804 erf = ffunction(float erf|erf|erf(float), <math.h>,"");
805
806
807 //-----`(ma.)erfc`-----
808 // Computes the complementary error function of the input signal.
809 //
810 // #### Usage
811 //
812 // ```
813 // _ : erfc : _
814 // ```
815 //
816 // Where:
817 //
818 // * `x`: input value
819 //
820 // #### Test
821 // ```
822 // ma = library("maths.lib");
823 // erfc_test = 0.5 : ma.erfc;
824 // ```
825 //-----
826 erfc = ffunction(float erfc|erfc|erfc(float), <math.h>,"");
827
828
829 //-----`(ma.)gamma`-----
830 // Computes the gamma function of the input signal.
831 //
832 // #### Usage
833 //
834 // ```
835 // _ : gamma : _
836 // ```
837 //
838 // Where:
839 //
840 // * `x`: positive input value
841 //
842 // #### Test
843 // ```
844 // ma = library("maths.lib");
845 // gamma_test = 3.0 : ma.gamma;
846 // ```
847 //-----
848 gamma = ffunction(float tgamma|tgamma|tgamma(float), <math.h>,"");
849
850
851 //-----`(ma.)lgamma`-----
852 // Calculates the natural logarithm of the absolute value of
853 // the gamma function of the input signal.
854 //
855 // #### Usage

```

```

856 //
857 // \
858 // : lgamma : _
859 // \
860 //
861 // Where:
862 //
863 // * `x`: positive input value
864 //
865 // #### Test
866 // \
867 // ma = library("maths.lib");
868 // lgamma_test = 3.0 : ma.lgamma;
869 // \
870 //-----
871 lgamma = ffunction(float lgammaf|lgamma|lgammal(float), <math.h>,"");
872
873
874 //-----`(ma.)J0`-----
875 // Computes the Bessel function of the first kind of order 0
876 // of the input signal.
877 //
878 // #### Usage
879 //
880 // \
881 // : J0 : _
882 // \
883 //
884 // Where:
885 //
886 // * `x`: input value
887 //
888 // #### Test
889 // \
890 // ma = library("maths.lib");
891 // J0_test = 1.0 : ma.J0;
892 // \
893 //-----
894 J0 = ffunction(float j0(float), <math.h>,"");
895
896
897 //-----`(ma.)J1`-----
898 // Computes the Bessel function of the first kind of order 1
899 // of the input signal.
900 //
901 // #### Usage
902 //
903 // \
904 // : J1 : _
905 // \
906 //
907 // Where:
908 //
909 // * `x`: input value
910 //
911 // #### Test
912 // \
913 // ma = library("maths.lib");
914 // J1_test = 1.0 : ma.J1;
915 // \
916 //-----
917 J1 = ffunction(float j1(float), <math.h>,"");
918
919
920 //-----`(ma.)Jn`-----
921 // Computes the Bessel function of the first kind of order n
922 // (first input signal) of the second input signal.
923 //

```

```

924 // #### Usage
925 //
926 // ...
927 // _ : Jn : _
928 // ...
929 //
930 // Where:
931 //
932 // * `n`: integer order
933 // * `x`: input value
934 //
935 // #### Test
936 // ...
937 // ma = library("maths.lib");
938 // Jn_test = (2, 1.0) : ma.Jn;
939 // ...
940 //-----
941 Jn = ffunction(float jn(int, float), <math.h>,"");
942
943
944 //-----`(ma.)Y0`-----
945 // Computes the linearly independent Bessel function of the second kind
946 // of order 0 of the input signal.
947 //
948 // #### Usage
949 //
950 // ...
951 // _ : Y0 : _
952 // ...
953 //
954 // Where:
955 //
956 // * `x`: positive input value
957 //
958 // #### Test
959 // ...
960 // ma = library("maths.lib");
961 // Y0_test = 1.0 : ma.Y0;
962 // ...
963 //-----
964 Y0 = ffunction(float y0(float), <math.h>,"");
965
966
967 //-----`(ma.)Y1`-----
968 // Computes the linearly independent Bessel function of the second kind
969 // of order 1 of the input signal.
970 //
971 // #### Usage
972 //
973 // ...
974 // _ : Y0 : _
975 // ...
976 //
977 // Where:
978 //
979 // * `x`: positive input value
980 //
981 // #### Test
982 // ...
983 // ma = library("maths.lib");
984 // Y1_test = 1.0 : ma.Y1;
985 // ...
986 //-----
987 Y1 = ffunction(float y1(float), <math.h>,"");
988
989
990 //-----`(ma.)Yn`-----
991 // Computes the linearly independent Bessel function of the second kind

```

```

992 // of order n (first input signal) of the second input signal.
993 //
994 // #### Usage
995 //
996 // ```
997 // _ : Yn : _
998 // ```
999 //
1000 // Where:
1001 //
1002 // * `n`: integer order
1003 // * `x`: positive input value
1004 //
1005 // #### Test
1006 // ```
1007 // ma = library("maths.lib");
1008 // Yn_test = (2, 1.0) : ma.Yn;
1009 // ```
1010 //-----
1011 Yn = ffunction(float yn(int, float), <math.h>,"");
1012
1013
1014 //-----`(ma.)fabs`, `(ma.)fmax`, `(ma.)fmin`
1015 // Just for compatibility...
1016 //
1017 // ```
1018 // fabs = abs
1019 // fmax = max
1020 // fmin = min
1021 // ```
1022 //-----
1023 fabs = abs;
1024 fmax = max;
1025 fmin = min;
1026
1027 //-----`(ma.)np2`-----
1028 // Gives the next power of 2 of x.
1029 //
1030 // #### Usage
1031 //
1032 // ```
1033 // np2(n) : _
1034 // ```
1035 //
1036 // Where:
1037 //
1038 // * `n`: an integer
1039 //
1040 // #### Test
1041 // ```
1042 // ma = library("maths.lib");
1043 // np2_test = 5 : ma.np2;
1044 // ```
1045 //-----
1046 np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);
1047
1048
1049 //-----`(ma.)frac`-----
1050 // Gives the fractional part of n.
1051 //
1052 // #### Usage
1053 //
1054 // ```
1055 // frac(n) : _
1056 // ```
1057 //
1058 // Where:

```



```

1059 //
1060 // * `n`: a decimal number
1061 //
1062 // #### Test
1063 // ```
1064 // ma = library("maths.lib");
1065 // frac_test = 3.75 : ma.frac;
1066 // ```
1067 //-----
1068 frac(n) = n - floor(n);
1069 decimal = frac;
1070
1071 // NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
1072 // in filters.lib so we decided to keep that one... decimal is declared though for
1073 // backward compatibility.
1074 // decimal(n) = n - floor(n);
1075
1076
1077 //-----`(ma.)modulo`-----
1078 // Modulus operation using the `(x%y+y)%y` formula to ensures the result is always non-
// negative, even if `x` is negative.
1079 //
1080 // #### Usage
1081 //
1082 // ```
1083 // modulo(x,y) : _
1084 // ```
1085 //
1086 // Where:
1087 //
1088 // * `x`: the numerator
1089 // * `y`: the denominator
1090 //
1091 // #### Test
1092 // ```
1093 // ma = library("maths.lib");
1094 // modulo_test = (-3, 4) : ma.modulo;
1095 // ```
1096 //-----
1097 modulo(x,y) = (x % y + y) % y;
1098
1099
1100 //-----`(ma.)isnan`-----
1101 // Return non-zero if x is a NaN.
1102 //
1103 // #### Usage
1104 //
1105 // ```
1106 // isnan(x)
1107 // _ : isnan : _
1108 // ```
1109 //
1110 // Where:
1111 //
1112 // * `x`: signal to analyse
1113 //
1114 // #### Test
1115 // ```
1116 // ma = library("maths.lib");
1117 // os = library("oscillators.lib");
1118 // isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
1119 // ```
1120 //-----
1121 isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");
1122
1123
1124 //-----`(ma.)isinf`-----
1125 // Return non-zero if x is a positive or negative infinity.

```

```

1126 //
1127 // #### Usage
1128 //
1129 // ...
1130 // isinf(x)
1131 // _ : isinf : _
1132 // ...
1133 //
1134 // Where:
1135 //
1136 // * `x`: signal to analyse
1137 //
1138 // #### Test
1139 // ...
1140 // ma = library("maths.lib");
1141 // os = library("oscillators.lib");
1142 // isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
1143 // ...
1144 //-----
1145 isinf = ffunction(int isinff|isinfl(float), <math.h>, "");
1146
1147 nextafter = ffunction(float nextafter(float, float), <math.h>, "");
1148
1149
1150 //-----`(ma.)chebychev`-----
1151 // Chebychev transformation of order N.
1152 //
1153 // #### Usage
1154 //
1155 // ...
1156 // _ : chebychev(N) : _
1157 // ...
1158 //
1159 // Where:
1160 //
1161 // * `N`: the order of the polynomial, a constant numerical expression
1162 //
1163 // #### Semantics
1164 //
1165 // ...
1166 //  $T[0](x) = 1$ ,
1167 //  $T[1](x) = x$ ,
1168 //  $T[n](x) = 2x*T[n-1](x) - T[n-2](x)$ 
1169 // ...
1170 //
1171 // #### Test
1172 // ...
1173 // ma = library("maths.lib");
1174 // chebychev_test = 0.5 : ma.chebychev(3);
1175 // ...
1176 //
1177 // #### References
1178 //
1179 // * <http://en.wikipedia.org/wiki/Chebyshev\_polynomial>
1180 //-----
1181 chebychev(0,x) = 1;
1182 chebychev(1,x) = x;
1183 chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);
1184
1185
1186 //-----`(ma.)chebypoly`-----
1187 // Linear combination of the first Chebyshev polynomials.
1188 //
1189 // #### Usage
1190 //
1191 // ...
1192 // _ : chebypoly((c0,c1,...,cn)) : _
1193 // ...

```

```

1194 //
1195 // Where:
1196 //
1197 // * `cn`: the different Chebychevs polynomials such that:
1198 // chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
1199 //
1200 // #### Test
1201 // ```
1202 // ma = library("maths.lib");
1203 // chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));
1204 // ```
1205 //
1206 // #### References
1207 //
1208 // <http://www.csounds.com/manual/html/chebyshevpoly.html>
1209 //-----
1210 chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
1211   with {
1212     L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
1213     L(n,c)      = chebychev(n)*c;
1214   };
1215
1216
1217 //-----`(ma.)diffn`-----
1218 // Negated first-order difference.
1219 //
1220 // #### Usage
1221 //
1222 // ```
1223 // _ : diffn : _
1224 // ```
1225 //
1226 // Where:
1227 //
1228 // * `x`: input signal
1229 //
1230 // #### Test
1231 // ```
1232 // ma = library("maths.lib");
1233 // os = library("oscillators.lib");
1234 // diffn_test = os.osc(440) : ma.diffn;
1235 // ```
1236 //-----
1237 diffn(x) = x' - x; // negated first-order difference
1238
1239
1240 //-----`(ma.)signum`-----
1241 // The signum function signum(x) is defined as
1242 // -1 for x<0, 0 for x==0, and 1 for x>0.
1243 //
1244 // #### Usage
1245 //
1246 // ```
1247 // _ : signum : _
1248 // ```
1249 //
1250 // Where:
1251 //
1252 // * `x`: input value
1253 //
1254 // #### Test
1255 // ```
1256 // ma = library("maths.lib");
1257 // signum_test = (-5.0) : ma.signum;
1258 // ```
1259 //-----
1260 signum(x) = (x>0)-(x<0);
1261

```

```

1262 //-----`(ma.)nextpow2`-----
1263 // The nextpow2(x) returns the lowest integer m such that
1264 // 2m >= x.
1265 //
1266 //
1267 // #### Usage
1268 //
1269 //
1270 // 2nextpow2(n) : _
1271 //
1272 // Useful for allocating delay lines, e.g.,
1273 //
1274 // delay(2nextpow2(maxDelayNeeded), currentDelay);
1275 //
1276 //
1277 // Where:
1278 //
1279 // * `n`: positive value whose next power-of-two exponent is computed
1280 //
1281 // #### Test
1282 //
1283 // ma = library("maths.lib");
1284 // nextpow2_test = 10.0 : ma.nextpow2;
1285 //
1286 //-----
1287 nextpow2(x) = ceil(log(x)/log(2.0));
1288
1289 //-----`(ma.)zc`-----
1290 // Indicator function for zero-crossing: it returns 1 if a zero-crossing
1291 // occurs, 0 otherwise.
1292 //
1293 //
1294 // #### Usage
1295 //
1296 //
1297 // _ : zc : _
1298 //
1299 //
1300 // Where:
1301 //
1302 // * `x`: input signal to monitor for zero crossings
1303 //
1304 // #### Test
1305 //
1306 // ma = library("maths.lib");
1307 // os = library("oscillators.lib");
1308 // zc_test = os.osc(440) : ma.zc;
1309 //
1310 //-----
1311 zc = ba.tAndH(!=(0)) <: *(mem) < 0;
1312
1313 //-----`(ma.)unwrap`-----
1314 // Unwrap the input signal so that successive output values never differ
1315 // by more than `pi`, switching to a 2*pi-complementary value when needed.
1316 //
1317 // #### Usage
1318 //
1319 //
1320 // _ : unwrap(pi) : _
1321 //
1322 //
1323 // Where:
1324 //
1325 // * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
1326 //
1327 // #### Test
1328 //
1329 // ma = library("maths.lib");

```

```

1330 // os = library("oscillators.lib");
1331 // unwrap_test = os.oscrc(100) : ma.unwrap(ma.PI);
1332 // ---
1333 //
1334 // #### Example test program
1335 //
1336 // ---
1337 // process = 0 - os.oscrc(1000) // the true phase is either -PI or +PI
1338 // : an.resonator(1,1000) : !, _ // oscillates between -PI and +PI
1339 // : ma.unwrap(ma.PI); // oscillates near +PI
1340 // ---
1341 //
1342 //-----
1343 unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
1344   dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi), -(2*pi));
1345   wrap(x,d) = x - d * floor(x / d);
1346 };
1347
1348 //-----`ma.)primes`-----
1349 // Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
1350 // primes(1) is 3, and so on. The waveform is length 2048, so the largest
1351 // precomputed prime is primes(2047) which is 17863.
1352 //
1353 // #### Usage
1354 //
1355 // ---
1356 // _ : primes : _
1357 // ---
1358 //
1359 // Where:
1360 //
1361 // * `x` : index of the prime number sequence (0-based).
1362 //
1363 // #### Test
1364 // ---
1365 // ma = library("maths.lib");
1366 // primes_test = 10 : ma.primes;
1367 // ---
1368 //-----
1369 primes(x) = rdtbl(waveform{
1370   2,3,5,7,11,13,17,19,23,29,
1371   31,37,41,43,47,53,59,61,67,71,
1372   73,79,83,89,97,101,103,107,109,113,
1373   127,131,137,139,149,151,157,163,167,173,
1374   179,181,191,193,197,199,211,223,227,229,
1375   233,239,241,251,257,263,269,271,277,281,
1376   283,293,307,311,313,317,331,337,347,349,
1377   353,359,367,373,379,383,389,397,401,409,
1378   419,421,431,433,439,443,449,457,461,463,
1379   467,479,487,491,499,503,509,521,523,541,
1380   547,557,563,569,571,577,587,593,599,601,
1381   607,613,617,619,631,641,643,647,653,659,
1382   661,673,677,683,691,701,709,719,727,733,
1383   739,743,751,757,761,769,773,787,797,809,
1384   811,821,823,827,829,839,853,857,859,863,
1385   877,881,883,887,907,911,919,929,937,941,
1386   947,953,967,971,977,983,991,997,1009,1013,
1387   1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1388   1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1389   1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1390   1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1391   1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1392   1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1393   1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1394   1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1395   1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1396   1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1397   1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,

```

1398 1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1399 1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1400 1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
1401 2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
1402 2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
1403 2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
1404 2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
1405 2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
1406 2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
1407 2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
1408 2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
1409 2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
1410 2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
1411 2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
1412 2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
1413 3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
1414 3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
1415 3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
1416 3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
1417 3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
1418 3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
1419 3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
1420 3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
1421 3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
1422 3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
1423 3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
1424 3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
1425 4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
1426 4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
1427 4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
1428 4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
1429 4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
1430 4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
1431 4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
1432 4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
1433 4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
1434 4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
1435 4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
1436 4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
1437 5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
1438 5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
1439 5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
1440 5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
1441 5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
1442 5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
1443 5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
1444 5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
1445 5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
1446 5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
1447 5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
1448 5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
1449 6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
1450 6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
1451 6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
1452 6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
1453 6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
1454 6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
1455 6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
1456 6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
1457 6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
1458 6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
1459 6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
1460 7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
1461 7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
1462 7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
1463 7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
1464 7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
1465 7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,

1466 7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
1467 7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
1468 7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
1469 7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
1470 7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
1471 8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
1472 8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
1473 8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
1474 8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
1475 8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
1476 8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
1477 8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
1478 8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
1479 8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
1480 8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
1481 8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
1482 9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
1483 9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
1484 9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
1485 9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
1486 9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
1487 9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
1488 9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
1489 9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
1490 9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
1491 9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
1492 9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
1493 10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
1494 10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
1495 10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
1496 10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
1497 10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
1498 10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
1499 10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
1500 10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
1501 10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
1502 10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
1503 10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
1504 11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
1505 11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
1506 11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
1507 11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
1508 11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
1509 11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
1510 11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
1511 11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
1512 11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
1513 11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
1514 12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
1515 12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
1516 12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
1517 12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
1518 12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
1519 12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
1520 12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
1521 12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
1522 12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
1523 12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
1524 12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
1525 13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
1526 13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
1527 13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
1528 13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
1529 13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
1530 13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
1531 13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
1532 13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
1533 13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,

```

1534 13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
1535 13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
1536 14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
1537 14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
1538 14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
1539 14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
1540 14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
1541 14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
1542 14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
1543 14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
1544 14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
1545 14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
1546 15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
1547 15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
1548 15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
1549 15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
1550 15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
1551 15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
1552 15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
1553 15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
1554 15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
1555 15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
1556 15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
1557 16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
1558 16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
1559 16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
1560 16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
1561 16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
1562 16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
1563 16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
1564 16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
1565 16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
1566 17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
1567 17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
1568 17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
1569 17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
1570 17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
1571 17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
1572 17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
1573 17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
1574 17789,17791,17807,17827,17837,17839,17851,17863
1575 }, x);

```

Listing 6: seam.dwt.lib

```

1 declare name "SEAM Discrete Wavelet Transform Library";
2 declare version "0.1";
3 declare author "Giuseppe Silvi";
4 declare license "CC3";
5 //
6 // SEAM Discrete Wavelet Transform Library
7 //
8 // Running-filter implementation of the Haar wavelet transform,
9 // operating sample-by-sample without downsampling. Suitable for
10 // real-time audio processing and spatial decomposition.
11 //
12 // The Haar wavelet is the simplest wavelet: a 2-point DFT applied
13 // as a running filter pair (Quadrature Mirror Filter):
14 //
15 // sum(x) = (x[n] + x[n-1]) / 2 -- lowpass (DC component)
16 // dif(x) = (x[n] - x[n-1]) / 2 -- highpass (AC component)
17 //
18 // Applied recursively, the QMF bank decomposes a signal into
19 // frequency-ordered sub-bands. The reversed form (dft2r) outputs

```



```

20 // highpass first, producing sub-bands ordered from highest to
21 // lowest frequency.
22 //
23 //
24 //
25 import("seam.lib");
26 //
27 sdw = library("seam.dwt.lib");
28 //
29 //          2-point DFT (Haar wavelet)
30 //
31 // Running sum/difference filter pair: the simplest QMF.
32 // Standard form: lowpass first, highpass second.
33 //
34 dft2(x) = s(x), d(x)
35 with {
36   s(x) = (x + x') / 2; // lowpass (DC)
37   d(x) = (x - x') / 2; // highpass (AC)
38 };
39 //
40 // Reversed form: highpass first, lowpass second.
41 // Used in the QMF bank so that outputs are ordered high-to-low,
42 // matching the ACN channel convention for spatial decomposition.
43 //
44 dft2r(x) = dft2(x) : ro.cross(2);
45 //
46 //          Recursive QMF filter bank
47 //
48 // qmf(N) applies dft2r recursively N times, producing N+1 outputs.
49 // At each level, the highpass component exits and the lowpass
50 // continues into the next decomposition. The result is a set of
51 // sub-band signals ordered from highest to lowest frequency.
52 //
53 qmf(0) = _;
54 qmf(1) = dft2r;
55 qmf(N) = dft2r : _, qmf(N-1);
56 //
57 //          Haar spatial decomposition
58 //
59 // haarmn(N): mono input to (N+1)^2 spatial components.
60 // The passthrough (unfiltered signal) occupies channel 0,
61 // followed by the QMF sub-bands as directional components
62 // ordered from highpass to lowpass.
63 //
64 // For Ambisonics spatial encoding:
65 // haarmn(0) = 1 output (mono, no decomposition)
66 // haarmn(1) = 4 outputs (first order)
67 // haarmn(2) = 9 outputs (second order)
68 // haarmn(3) = 16 outputs (third order)
69 //
70 haarmn(0) = _;
71 haarmn(N) = _ <: _, qmf((N+1)^2-2);

```

Listing 7: signals.lib

```

1 //##### signals.lib #####
2 // Signals library. Its official prefix is `si`.
3 //
4 // This library provides fundamental signal processing operations for Faust,
5 // including generators, combinators, selectors, and basic DSP utilities. It defines
6 // essential functions used across all Faust libraries for building and manipulating
7 // audio and control signals.
8 //
9 // The Signals library is organized into 1 section:

```

```

10 //
11 // * [Functions Reference](#functions-reference)
12 //
13 // #### References
14 //
15 // * <https://github.com/grame-cncm/faustlibraries/blob/master/signals.lib>
16 //#####
17
18 /*****
19 *****/
20 FAUST library file, GRAME section
21
22 Except where noted otherwise, Copyright (C) 2003-2017 by GRAME,
23 Centre National de Creation Musicale.
24
25 -----
26 GRAME LICENSE
27
28 This program is free software; you can redistribute it and/or modify
29 it under the terms of the GNU Lesser General Public License as
30 published by the Free Software Foundation; either version 2.1 of the
31 License, or (at your option) any later version.
32
33 This program is distributed in the hope that it will be useful,
34 but WITHOUT ANY WARRANTY; without even the implied warranty of
35 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
36 GNU Lesser General Public License for more details.
37
38 You should have received a copy of the GNU Lesser General Public
39 License along with the GNU C Library; if not, write to the Free
40 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
41 02111-1307 USA.
42
43 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
44 larger FAUST program which directly or indirectly imports this library
45 file and still distribute the compiled code generated by the FAUST
46 compiler, or a modified version of this compiled code, under your own
47 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
48 grants you the right to freely choose the license for the resulting
49 compiled code. In particular the resulting compiled code has no obligation
50 to be LGPL or GPL. For example you are free to choose a commercial or
51 closed source license or any other license if you decide so.
52 *****/
53 *****/
54
55 ba = library("basics.lib");
56 ro = library("routes.lib");
57 ma = library("maths.lib");
58 si = library("signals.lib");
59
60 declare name "Faust Routing Library";
61 declare version "1.6.0";
62
63 //=====Functions Reference=====
64 //=====
65
66 //-----`(si.)bus`-----
67 // Put N cables in parallel.
68 // `bus` is a standard Faust function.
69 //
70 // #### Usage
71 //
72 // ```
73 // bus(N)
74 // bus(4) : _,-,-_
75 // ```
76 //
77 // Where:

```

```

78 //
79 // * `N`: is an integer known at compile time that indicates the number of parallel cables
80 //
81 // #### Test
82 // ```
83 // si = library("signals.lib");
84 // bus_test = (
85 //   hslider("bus:x0", 0.25, -1, 1, 0.01),
86 //   hslider("bus:x1", -0.5, -1, 1, 0.01),
87 //   hslider("bus:x2", 0.75, -1, 1, 0.01)
88 // ) : si.bus(3);
89 // ```
90 //-----
91 bus(0) = 0!;
92 bus(1) = _; // avoids a lot of "bus(1)" labels in block diagrams
93 bus(2) = _,_; // avoids a lot of "bus(2)" labels in block diagrams
94 bus(N) = par(i, N, _);
95
96 //-----`(si.)block`-----
97 // Block - terminate N signals.
98 // `block` is a standard Faust function.
99 //
100 // #### Usage
101 // ```
102 // ```
103 // si.bus(N) : block(N)
104 // ```
105 //
106 // Where:
107 //
108 // * `N`: the number of signals to be blocked known at compile time
109 //
110 // #### Test
111 // ```
112 // si = library("signals.lib");
113 // block_test = (
114 //   hslider("block:x0", 0.5, -1, 1, 0.01),
115 //   hslider("block:x1", -0.25, -1, 1, 0.01)
116 // ) : (si.block(1), _);
117 // ```
118 //-----
119 block(N) = par(i, N, !);
120
121 //-----`(si.)interpolate`-----
122 // Linear interpolation between two signals.
123 //
124 // #### Usage
125 // ```
126 // ```
127 // _ : interpolate(i) : _
128 // ```
129 //
130 // Where:
131 //
132 // * `i`: interpolation control between 0 and 1 (0: first input; 1: second input)
133 //
134 // #### Test
135 // ```
136 // si = library("signals.lib");
137 // os = library("oscillators.lib");
138 // interpolate_test = si.interpolate(
139 //   hslider("interpolate:mix", 0.5, 0, 1, 0.01),
140 //   os.osc(220),
141 //   os.osc(440)
142 // );
143 // ```
144 //-----
145 interpolate(i,x,y) = x + i*(y-x);

```

```

146 //-----`(si.)repeat`-----
147 // Repeat an effect N time(s) and take the parallel sum of all
148 // intermediate buses.
149 //
150 //
151 // #### Usage
152 //
153 //
154 // si.bus(inputs(FX)) : repeat(N, FX) : si.bus(outputs(FX))
155 //
156 //
157 // Where:
158 //
159 // * `N`: Number of repetitions, minimum of 1, a constant numerical expression
160 // * `FX`: an arbitrary effect (N inputs and N outputs) that will be repeated
161 //
162 // #### Test
163 //
164 // si = library("signals.lib");
165 // repeat_test = hslider("repeat:input", 0.5, -1, 1, 0.01) : si.repeat(3, *(0.5));
166 //
167 //
168 // Example 1:
169 //
170 // process = repeat(2, dm.zita_light) : *.5, *.5;
171 //
172 //
173 // Example 2:
174 //
175 // N = 4;
176 // C = 2;
177 // fx(i) = i+1, par(j, C, @(i*5000));
178 // process = 0, si.bus(C) : repeat(N, fx) : !, par(i, C, *.2/N);
179 //
180 //
181 // #### References
182 //
183 // * <https://github.com/orlarey/presentation-compileur-faust/blob/master/slides.pdf>
184 //-----
185 declare repeat author "Yann Orlarey and Stéphane Letz, revised by David Braun";
186
187 repeat(1, FX) = FX;
188 repeat(n, FX) = FX <: si.bus(N), repeat(n-1, FX) :> si.bus(N)
189 with {
190   N = outputs(FX);
191 };
192
193 //-----`(si.)smoo`-----
194 // Smoothing function based on `smooth` ideal to smooth UI signals
195 // (sliders, etc.) down. Approximately, this is a 7 Hz one-pole
196 // low-pass considering the coefficient calculation:
197 //   exp(-2pi*CF/SR).
198 //
199 // `smoo` is a standard Faust function.
200 //
201 // #### Usage
202 //
203 //
204 // hslider(...) : smoo;
205 //
206 //
207 // #### Test
208 //
209 // si = library("signals.lib");
210 // smoo_test = hslider("smoo:input", 0.5, -1, 1, 0.01) : si.smoo;
211 //
212 //-----
213 smoo = si.smooth(1 - 44.1/ma.SR);

```

```

214 //-----`(si.)polySmooth`-----
215 // A smoothing function based on `smooth` that doesn't smooth when a
216 // trigger signal is given. This is very useful when making
217 // polyphonic synthesizer to make sure that the value of the parameter
218 // is the right one when the note is started.
219 //
220 //
221 // #### Usage
222 //
223 // ```
224 // hslider(...) : polySmooth(g,s,d) : _
225 // ```
226 //
227 // Where:
228 //
229 // * `g`: the gate/trigger signal used when making polyphonic synths
230 // * `s`: the smoothness (see `smooth`)
231 // * `d`: the number of samples to wait before the signal start being
232 //   smoothed after `g` switched to 1
233 //
234 // #### Test
235 // ```
236 // si = library("signals.lib");
237 // polySmooth_test = hslider("polySmooth:input", 0.5, -1, 1, 0.01)
238 // : si.polySmooth(button("polySmooth:gate"), 0.999, 32);
239 // ```
240 //-----
241 polySmooth(g,s,d) = si.smooth(s*((g==(g@d)) | (g == 0)));
242
243 //-----`(si.)smoothAndH`-----
244 // A smoothing function based on `smooth` that holds its output
245 // signal when a trigger is sent to it. This feature is convenient
246 // when implementing polyphonic instruments to prevent some
247 // smoothed parameter to change when a note-off event is sent.
248 //
249 // #### Usage
250 //
251 // ```
252 // hslider(...) : smoothAndH(g,s) : _
253 // ```
254 //
255 // Where:
256 //
257 // * `g`: the hold signal (0 for hold, 1 for bypass)
258 // * `s`: the smoothness (see `smooth`)
259 //
260 // #### Test
261 // ```
262 // si = library("signals.lib");
263 // smoothAndH_test = hslider("smoothAndH:input", 0.5, -1, 1, 0.01)
264 // : si.smoothAndH(button("smoothAndH:hold"), 0.999);
265 // ```
266 //-----
267 smoothAndH(t,s) = si.smooth(s*t) : ba.sAndH(t);
268
269 //-----`(si.)bsmooth`-----
270 // Block smooth linear interpolation during a block of samples (given by the `ma.BS` value).
271 //
272 // #### Usage
273 //
274 // ```
275 // hslider(...) : bsmooth : _
276 // ```
277 //
278 // #### Test
279 // ```
280 // si = library("signals.lib");
281 // bsmooth_test = hslider("bsmooth:input", 0.5, -1, 1, 0.01) : si.bsmooth;

```

```

282 // ---
283 //-----
284 bsmooth(c) = +(i) ~ _
285 with {
286     i = (c-c@n)/n;
287     n = min(4096, max(1, ma.BS));
288 };
289
290 //-----`si.`dot`-----
291 // Dot product for two vectors of size N.
292 //
293 // ### Usage
294 //
295 // ---
296 // si.bus(N), si.bus(N) : dot(N) : _
297 // ---
298 //
299 // Where:
300 //
301 // * `N`: size of the vectors (int, must be known at compile time)
302 //
303 // ### Test
304 // ---
305 // si = library("signals.lib");
306 // os = library("oscillators.lib");
307 // dot_test = (
308 //     os.osc(100), os.osc(200), os.osc(300),
309 //     os.osc(400), os.osc(500), os.osc(600)
310 // ) : si.dot(3);
311 // ---
312 //-----
313 dot(N) = ro.interleave(N,2) : par(i,N,*) :> _;
314
315 // end GRAME section
316 //#####
317 //*****
318 FAUST library file, jos section
319
320 Except where noted otherwise, The Faust functions below in this
321 section are Copyright (C) 2003-2017 by Julius O. Smith III <jos@ccrma.stanford.edu>
322 ([jos](http://ccrma.stanford.edu/~jos/)), and released under the
323 (MIT-style) [STK-4.3](#stk-4.3-license) license.
324
325 All Markdown comments in this section are Copyright 2016-2017 by Romain
326 Michon and Julius O. Smith III, and are released under the
327 [CCIA4I](https://creativecommons.org/licenses/by/4.0/) license (TODO: if/when Romain agrees!)
328
329 *****/
330 //-----`si.`smooth`-----
331 // Exponential smoothing by a unity-dc-gain one-pole lowpass.
332 // `smooth` is a standard Faust function.
333 //
334 // ### Usage:
335 //
336 // ---
337 // _ : si.smooth(ba.tau2pole(tau)) : _
338 // ---
339 //
340 // Where:
341 //
342 // * `tau`: desired smoothing time constant in seconds, or
343 //
344 // ---
345 // hslider(...) : smooth(s) : _
346 // ---
347 //
348 // Where:
349 //

```

```

350 // * `s`: smoothness between 0 and 1. s=0 for no smoothing, s=0.999 is "very smooth",
351 // s>1 is unstable, and s=1 yields the zero signal for all inputs.
352 // The exponential time-constant is approximately 1/(1-s) samples, when s is close to
353 // (but less than) 1.
354 //
355 // #### Test
356 // ```
357 // si = library("signals.lib");
358 // smooth_test = hslider("smooth:input", 0.5, -1, 1, 0.01) : si.smooth(0.9);
359 // ```
360 //
361 // #### References
362 //
363 // * <https://ccrma.stanford.edu/~jos/mdft/Convolution\_Example\_2\_ADSR.html>
364 // * <https://ccrma.stanford.edu/~jos/aspf/Appendix\_B\_Inspecting\_Assembly.html>
365 //-----
366
367 // See [grame-cncm/faustlibraries]: Minor improvement to si.smoo. (Discussion #106)
368
369 smooth_imp = case {
370   // y[n] = (1 - s) * x[n] + s * y[n - 1]
371   (0,s) => \x.(fb ~ _ with { fb(y) = (1.0 - s) * x + s * y; });
372
373   // y[n] = s * (y[n - 1] - x[n]) + x[n]
374   (1,s) => \x.(fb ~ _ with { fb(y) = s * (y - x) + x; });
375
376   // y[n] = y[n - 1] + (1 - s) * (x[n] - y[n - 1])
377   (2,s) => \x.(fb ~ _ with { fb(y) = y + (1.0 - s) * (x - y); });
378 };
379
380 // The best compromise on modern CPUs where two independent multiplications can be done in
381 // parallel.
382 // Other versions could possibly be faster on embedded devices.
383 smooth = smooth_imp(0);
384
385 //-----`si.`smoothq`-----
386 // Smoothing with continuously variable curves from Exponential to Linear, with a constant
387 // time.
388 //
389 // #### Usage
390 //
391 // ```
392 // _ : smoothq(time, q) : _;
393 // ```
394 //
395 // Where:
396 //
397 // * `time`: seconds to reach target
398 // * `q`: curve shape (between 0..1, 0 is Exponential, 1 is Linear)
399 //
400 // #### Test
401 // ```
402 // si = library("signals.lib");
403 // smoothq_test = hslider("smoothq:input", 0.5, -1, 1, 0.01) : si.smoothq(0.25, 0.5);
404 // ```
405 //-----
406 declare smoothq author "Andrew John March";
407 declare smoothq licence "STK-4.3";
408
409 smoothq(time, q, tar) = envelope
410   with {
411     ratio = pow(q, 5) * 32 : max(0.001);
412     coef = (ratio / (1 + ratio))^(1 / (ma.SR * time));
413     fb(cBase, cTar, cSrc, cDistance, cDir, cY, cOut) = base, tar, src, distance, dir, y,
414       out
415     with {
416       trig = cTar != tar;

```

```

415     src = select2(trig, cSrc, cOut);
416     dir = select2(trig, cDir, tar > src);
417     base = select2(trig, cBase, select2(dir,
418         (0.0 - ratio) * (1.0 - coef),
419         (1.0 + ratio) * (1.0 - coef)
420     ));
421     distance = select2(trig, cDistance, abs(tar - src));
422     y = select2(dir,
423         select2(trig,
424             max( 0, base + cY * coef),
425             1
426         ),
427         select2(trig,
428             min( 1, base + cY * coef),
429             0
430         )
431     );
432     out = select2(dir,
433         tar + y * distance,
434         src + y * distance
435     );
436 };
437 envelope = fb ~ (_,_,_,_,_,_) : !,!,!,!,!,!,-;
438 };
439
440
441 //-----`si.)cbus`-----
442 // N parallel cables for complex signals.
443 // `cbus` is a standard Faust function.
444 //
445 // #### Usage
446 //
447 // ...
448 // cbus(N)
449 // cbus(4) : (r0,i0), (r1,i1), (r2,i2), (r3,i3)
450 // ...
451 //
452 // Where:
453 //
454 // * `N` is an integer known at compile time that indicates the number of parallel cables.
455 // * each complex number is represented by two real signals as (real,imag)
456 //
457 // #### Test
458 // ...
459 // si = library("signals.lib");
460 // os = library("oscillators.lib");
461 // cbus_test = (
462 //     os.osc(100), os.osc(150),
463 //     os.osc(200), os.osc(250)
464 // ) : si.cbush(2);
465 // ...
466 //-----
467 cbush(1) = (_,_);
468 cbush(N) = par(i, N, (_,_));
469
470 //-----`si.)cmul`-----
471 // Multiply two complex signals pointwise.
472 // `cmul` is a standard Faust function.
473 //
474 // #### Usage
475 //
476 // ...
477 // (r1,i1) : cmul(r2,i2) : (_,_)
478 // ...
479 //
480 // Where:
481 //
482 // * Each complex number is represented by two real signals as (real,imag), so

```



```

483 // - `(r1,i1)` = real and imaginary parts of signal 1
484 // - `(r2,i2)` = real and imaginary parts of signal 2
485 //
486 // #### Test
487 // ```
488 // si = library("signals.lib");
489 // os = library("oscillators.lib");
490 // cmul_test = si.cmul(
491 //   os.osc(110), os.osc(220),
492 //   os.osc(330), os.osc(440)
493 // );
494 // ```
495 //-----
496 cmul(r1,i1,r2,i2) = (r1*r2 - i1*i2), (r1*i2 + r2*i1);
497
498 //-----`(si.)cconj`-----
499 // Complex conjugation of a (complex) signal.
500 // `cconj` is a standard Faust function.
501 //
502 // #### Usage
503 //
504 // ```
505 // (r1,i1) : cconj : (_,_)
506 // ```
507 //
508 // Where:
509 //
510 // * Each complex number is represented by two real signals as (real,imag), so
511 // - `(r1,i1)` = real and imaginary parts of the input signal
512 // - `(r1,-i1)` = real and imaginary parts of the output signal
513 //
514 // #### Test
515 // ```
516 // si = library("signals.lib");
517 // os = library("oscillators.lib");
518 // cconj_test = (os.osc(210), os.osc(310)) : si.cconj;
519 // ```
520 //-----
521 cconj = _, *(-1);
522
523 // end jos section
524 /*****
525 FAUST library file, further contributions section
526 All contributions below should indicate both the contributor and terms
527 of license. If no such indication is found, "git blame" will say who
528 last edited each line, and that person can be emailed to inquire about
529 license disposition, if their license choice is not already indicated
530 elsewhere among the libraries. It is expected that all software will be
531 released under LGPL, STK-4.3, MIT, BSD, or a similar FOSS license.
532 *****/
533
534 //-----`(si.)onePoleSwitching`-----
535 // One pole filter with independent attack and release times.
536 //
537 // #### Usage
538 //
539 // ```
540 // _ : onePoleSwitching(att,rel) : _
541 // ```
542 //
543 // Where:
544 //
545 // * `att`: the attack tau time constant in second
546 // * `rel`: the release tau time constant in second
547 //
548 // #### Test
549 // ```
550 // si = library("signals.lib");

```

```

551 // onePoleSwitching_test = hslider("onePoleSwitching:input", 0.5, -1, 1, 0.01)
552 // : si.onePoleSwitching(0.05, 0.2);
553 // ...
554 //-----
555 declare onePoleSwitching author "Jonatan Liljedahl, revised by Dario Sanfilippo";
556 declare onePoleSwitching licence "STK-4.3";
557
558 onePoleSwitching(att, rel, x) = loop ~ _
559   with {
560     loop(yState) = (1.0 - coeff) * x + coeff * yState
561     with {
562       coeff = ba.if(x > yState, ba.tau2pole(att), ba.tau2pole(rel));
563     };
564   };
565
566 // Kept for backward compatibility
567 lag_ud = onePoleSwitching;
568
569 // end further further contributions section
570
571 //-----`si.)rev`-----
572 // Reverse the input signal by blocks of n>0 samples. `rev(1)` is the identity
573 // function. `rev(n)` has a latency of `n-1` samples.
574 //
575 // ### Usage
576 // ...
577 // _ : rev(n) : _
578 // ...
579 //
580 // Where:
581 //
582 // * `n`: the block size in samples
583 //
584 // ### Test
585 // ...
586 // si = library("signals.lib");
587 // os = library("oscillators.lib");
588 // rev_test = os.osc(440) : si.rev(32);
589 // ...
590 //-----
591 declare rev author "Yann Orlarey";
592
593 rev(n) = @((phase(n)*2)
594   with {
595     phase(n) = 1 : (+ : %(n)) ~ _ : max(0) : min(n-1);
596   });
597
598 //-----`si.)vecOp`-----
599 //
600 // This function is a generalisation of Faust's iterators such as `prod` and
601 // `sum`, and it allows to perform operations on an arbitrary number of
602 // vectors, provided that they all have the same length. Unlike Faust's
603 // iterators `prod` and `sum` where the vector size is equal to one and the
604 // vector space dimension must be specified by the user, this function will
605 // infer the vector space dimension and vector size based on the vectors list
606 // that we provide.
607 //
608 // The outputs of the function are equal to the vector size, whereas the
609 // number of inputs is dependent on whether the elements of the vectors
610 // provided expect an incoming signal themselves or not. We will see a
611 // clarifying example later; in general, the number of total inputs will
612 // be the sum of the inputs in each input vector.
613 //
614 // Note that we must provide a list of at least two vectors, each with a size
615 // that is greater or equal to one.
616 //
617 //
618 //

```

```

619 // #### Usage
620 //
621 // ...
622 // si.bus(inputs(vectorsList)) : vecOp((vectorsList), op) : si.bus(outputs(ba.take(1,
        vectorsList)));
623 // ...
624 //
625 // Where:
626 //
627 // * `vectorsList`: is a list of vectors
628 // * `op`: is a two-input, one-output operator
629 //
630 // #### Test
631 // ...
632 // si = library("signals.lib");
633 // vecOp_test = si.vecOp((v0, v1), +)
634 // with {
635 //   v0 = (hslider("vecOp:v0_0", 0.1, -1, 1, 0.01), hslider("vecOp:v0_1", 0.2, -1, 1, 0.01))
636 //   ;
637 //   v1 = (hslider("vecOp:v1_0", 0.3, -1, 1, 0.01), hslider("vecOp:v1_1", 0.4, -1, 1, 0.01))
638 //   ;
639 // };
640 // ...
641 //
642 // For example, consider the following vectors lists:
643 //
644 // v0 = (0 , 1 , 2 , 3);
645 // v1 = (4 , 5 , 6 , 7);
646 // v2 = (8 , 9 , 10 , 11);
647 // v3 = (12 , 13 , 14 , 15);
648 // v4 = (+ (16) , _ , 18 , *(19));
649 // vv = (v0 , v1 , v2 , v3);
650 //
651 // Although Faust has limitations for list processing, these vectors can be
652 // combined or processed individually.
653 //
654 // If we do:
655 //
656 // process = vecOp(v0, +);
657 //
658 // the function will deduce a vector space of dimension equal to four and
659 // a vector length equal to one. Note that this is equivalent to writing:
660 //
661 // process = v0 : sum(i, 4, _);
662 //
663 // Similarly, we can write:
664 //
665 // process = vecOp((v0 , v1), *) :> _;
666 //
667 // and we have a dimension-two space and length-four vectors. This is the dot
668 // product between vectors v0 and v1, which is equivalent to writing:
669 //
670 // process = v0 , v1 : dot(4);
671 //
672 // The examples above have no inputs, as none of the elements of the vectors
673 // expect inputs. On the other hand, we can write:
674 //
675 // process = vecOp((v4 , v4), +);
676 //
677 // and the function will have six inputs and four outputs, as each vector
678 // has three of the four elements expecting an input, times two, as the two
679 // input vectors are identical.
680 //
681 // Finally, we can write:
682 //
683 // process = vecOp(vv, &);
684 //
685 // to perform the bitwise AND on all the elements at the same position in

```

```

684 // each vector, having dimension equal to the vector length equal to four.
685 //
686 // Or even:
687 //
688 //     process = vecOp((vv , vv), &);
689 //
690 // which gives us a dimension equal to two, and a vector size equal to sixteen.
691 //
692 // For a more practical use-case, this is how we can implement a time-invariant
693 // feedback delay network with Hadamard matrix:
694 //
695 //     N = 4;
696 //     normalisation = 1.0 / sqrt(N);
697 //     coeffVec = par(i, N, .99 * normalisation);
698 //     delVec = par(i, N, (i + 1) * 3);
699 //     process = vecOp((si.bus(N) , si.bus(N)), +) ~
700 //         vecOp((vecOp((ro.hadamard(N) , coeffVec), *) , delVec), @);
701 //
702 //-----
703 declare vecOp author "Dario Sanfilippo";
704 declare vecOp copyright "Copyright (C) 2022 Dario Sanfilippo
705     <sanfilippo.dario@gmail.com>";
706 declare vecOp license "MIT License";
707 vecOp(vectorsList, op) =
708     vectorsList : seq(i, vecDim - 1, vecOp2D , vecBus(vecDim - 2 - i))
709     with {
710         vecBus(0) = par(i, vecLen, 0 : !);
711         vecBus(dim) = par(i, dim, si.bus(vecLen));
712         vecOp2D = ro.interleave(vecLen, 2) : par(i, vecLen, op);
713         vecDim = outputs(vectorsList) / vecLen;
714         vecLen = outputs(ba.take(1, vectorsList));
715     };
716
717 /*
718 Balanced par, sum, and prod, faster versions of built-in par, sum, prod
719 when the repeated expression doesn't depend on the variable.
720 */
721
722 //-----`si.)bpar`-----
723 // Balanced `par` where the repeated expression doesn't depend on a variable.
724 // The built-in `par` is implemented as an unbalanced tree, and also has
725 // to substitute the variable into the repeated expression, which is expensive
726 // even when the variable doesn't appear. This version is implemented as a
727 // balanced tree (which allows node reuse during tree traversal) and also
728 // doesn't search for the variable. This can be much faster than `par` to compile.
729 //
730 // #### Usage
731 //
732 // ```
733 // si.bus(N * inputs(f)) : bpar(N, f) : si.bus(N * outputs(f))
734 // ```
735 //
736 // Where:
737 //
738 // * `N`: number of repetitions, minimum 1, a constant numerical expression
739 // * `f`: an arbitrary expression
740 //
741 // #### Test
742 //
743 // ```
744 // si = library("signals.lib");
745 // os = library("oscillators.lib");
746 // bpar_test = (os.osc(120), os.osc(240), os.osc(360)) : si.bpar(3, *(0.5));
747 // ```
748 //
749 // Example:
750 //
751 // square each of 4000 inputs
752 // process = si.bpar(4000, (<: _, _ : *));

```

```

752 // ...
753 //
754 //-----
755 declare bpar author "Haggai Nuchi";
756 declare bpar copyright "Copyright (C) 2024 Haggai Nuchi <haggai@haggainuchi.com>";
757 declare bpar license "MIT License";
758 bpar(1, f) = f;
759 bpar(N, f) = bpar(left, f), bpar(right, f) with {
760     left = ma.np2(N) >> 1; // biggest power of 2 less than N
761     right = N - left;
762 };
763
764 //-----`(si.)bsum`-----
765 // Balanced `sum`, see `si.bpar`.
766 //
767 // ### Usage
768 //
769 // ...
770 // si.bus(N * inputs(f)) : bsum(N, f) : _
771 // ...
772 //
773 // Where:
774 //
775 // * `N`: number of repetitions, minimum 1, a constant numerical expression
776 // * `f`: an arbitrary expression with 1 output.
777 //
778 // ### Test
779 // ...
780 // si = library("signals.lib");
781 // os = library("oscillators.lib");
782 // bsum_test = (os.osc(100), os.osc(200), os.osc(300)) : si.bsum(3, *(0.5));
783 // ...
784 //
785 // Example:
786 // ...
787 // // square each of 1000 inputs and add the results
788 // process = si.bsum(1000, (_ <: _, _ : *));
789 // ...
790 //-----
791 declare bsum author "Haggai Nuchi";
792 declare bsum copyright "Copyright (C) 2024 Haggai Nuchi <haggai@haggainuchi.com>";
793 declare bsum license "MIT License";
794 bsum(1, f) = f;
795 bsum(N, f) = bsum(left, f) + bsum(right, f) with {
796     left = ma.np2(N) >> 1; // biggest power of 2 less than N
797     right = N - left;
798 };
799
800 //-----`(si.)bprod`-----
801 // Balanced `prod`, see `si.bpar`.
802 //
803 // ### Usage
804 //
805 // ...
806 // si.bus(N * inputs(f)) : bprod(N, f) : _
807 // ...
808 //
809 // Where:
810 //
811 // * `N`: number of repetitions, minimum 1, a constant numerical expression
812 // * `f`: an arbitrary expression with 1 output.
813 //
814 // ### Test
815 // ...
816 // si = library("signals.lib");
817 // bprod_test = (
818 //     hslider("bprod:x0", 0.5, 0, 2, 0.01),
819 //     hslider("bprod:x1", 0.8, 0, 2, 0.01)

```

```

820 // ) : si.bprod(2, _);
821 // ...
822 //
823 // Example:
824 // ...
825 // // Add 8000 consecutive inputs (in pairs) and multiply the results
826 // process = si.bprod(4000, +);
827 // ...
828 //-----
829 declare bprod author "Haggai Nuchi";
830 declare bprod copyright "Copyright (C) 2024 Haggai Nuchi <haggai@haggainuchi.com>";
831 declare bprod license "MIT License";
832 bprod(1, f) = f;
833 bprod(N, f) = bprod(left, f) * bprod(right, f) with {
834   left = ma.np2(N) >> 1; // biggest power of 2 less than N
835   right = N - left;
836 };

```

Listing 8: routes.lib

```

1 //##### routes.lib #####
2 // Routing library. Its official prefix is `ro`.
3 //
4 // This library provides tools for managing and organizing audio and control signal
5 // routing in Faust. It includes functions for channel mapping, splitting, merging, and
6 // dynamic routing, as well as utilities for building multichannel processing structures.
7 //
8 // The Routes library is organized into 1 section:
9 //
10 // * [Functions Reference](#functions-reference)
11 //
12 // ### References
13 //
14 // * <https://github.com/grame-cncm/faustlibraries/blob/master/routes.lib>
15 //#####
16
17 /*****
18 *****/
19 FAUST library file
20 Copyright (C) 2003-2019 GRAME, Centre National de Creation Musicale
21 -----
22 This program is free software; you can redistribute it and/or modify
23 it under the terms of the GNU Lesser General Public License as
24 published by the Free Software Foundation; either version 2.1 of the
25 License, or (at your option) any later version.
26
27 This program is distributed in the hope that it will be useful,
28 but WITHOUT ANY WARRANTY; without even the implied warranty of
29 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
30 GNU Lesser General Public License for more details.
31
32 You should have received a copy of the GNU Lesser General Public
33 License along with the GNU C Library; if not, write to the Free
34 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
35 02111-1307 USA.
36
37 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
38 larger FAUST program which directly or indirectly imports this library
39 file and still distribute the compiled code generated by the FAUST
40 compiler, or a modified version of this compiled code, under your own
41 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
42 grants you the right to freely choose the license for the resulting
43 compiled code. In particular the resulting compiled code has no obligation
44 to be LGPL or GPL. For example you are free to choose a commercial or

```

```

45 closed source license or any other license if you decide so.
46 ****
47 ****/
48
49 ba = library("basics.lib");
50 si = library("signals.lib");
51 sp = library("spats.lib");
52
53 declare name "Faust Signal Routing Library";
54 declare version "1.3.0";
55
56 //=====Functions Reference=====
57 //=====
58
59 //-----`(ro.)cross`-----
60 // Cross N signals: `(x1,x2,..,xn) -> (xn,..,x2,x1)`.
61 // `cross` is a standard Faust function.
62 //
63 // #### Usage
64 //
65 // ```
66 // cross(N)
67 // _,'- : cross(3) : _,'-
68 // ```
69 //
70 // Where:
71 //
72 // * `N`: number of signals (int, as a constant numerical expression)
73 //
74 // #### Test
75 // ```
76 // ro = library("routes.lib");
77 // os = library("oscillators.lib");
78 // cross_test = (os.osc(200), os.osc(300), os.osc(400)) : ro.cross(3);
79 // ```
80 //
81 // #### Note
82 //
83 // Special case: `cross2`:
84 //
85 // ```
86 // cross2 = _,cross(2),_;
87 // ```
88 //-----
89 // cross n cables : `(x1,x2,..,xn) -> (xn,..,x2,x1)`
90 cross(N) = route(N, N, par(i, N, (i+1, N-i)));
91 cross2 = _,cross(2),_; // for compatibility with some old misceffects.lib functions
92
93
94 //-----`(ro.)crossnn`-----
95 // Cross two `bus(N)`s.
96 //
97 // #### Usage
98 //
99 // ```
100 // (si.bus(2*N)) : crossnn(N) : (si.bus(2*N))
101 // ```
102 //
103 // Where:
104 //
105 // * `N`: the number of signals in the `bus` (int, as a constant numerical expression)
106 //
107 // #### Test
108 // ```
109 // ro = library("routes.lib");
110 // os = library("oscillators.lib");
111 // crossnn_test = (os.osc(110), os.osc(220), os.osc(330), os.osc(440)) : ro.crossnn(2);
112 // ```

```

```

113 //-----
114 crossnn(N) = crossNM(N,N);
115
116
117 //-----`(ro.)crossn1`-----
118 // Cross `bus(N)` and `bus(1)`.
119 //
120 // #### Usage
121 //
122 // ```
123 // (si.bus(N),_) : crossn1(N) : (_,si.bus(N))
124 // ```
125 //
126 // Where:
127 //
128 // * `N`: the number of signals in the first `bus` (int, as a constant numerical expression)
129 //
130 // #### Test
131 // ```
132 // ro = library("routes.lib");
133 // os = library("oscillators.lib");
134 // crossn1_test = (os.osc(100), os.osc(200), os.osc(300), os.osc(400)) : ro.crossn1(3);
135 // ```
136 //-----
137 crossn1(N) = crossNM(N,1);
138
139
140 //-----`(ro.)crossin`-----
141 // Cross `bus(1)` and `bus(N)`.
142 //
143 // #### Usage
144 //
145 // ```
146 // (_,si.bus(N)) : crossin(N) : (si.bus(N),_)
147 // ```
148 //
149 // Where:
150 //
151 // * `N`: the number of signals in the second `bus` (int, as a constant numerical expression)
152 //
153 // #### Test
154 // ```
155 // ro = library("routes.lib");
156 // os = library("oscillators.lib");
157 // crossin_test = (os.osc(150), os.osc(250), os.osc(350), os.osc(450)) : ro.crossin(3);
158 // ```
159 //-----
160 crossin(N) = crossNM(1,N);
161
162
163 //-----`(ro.)crossNM`-----
164 // Cross `bus(N)` and `bus(M)`.
165 //
166 // #### Usage
167 //
168 // ```
169 // (si.bus(N),si.bus(M)) : crossNM(N,M) : (si.bus(M),si.bus(N))
170 // ```
171 //
172 // Where:
173 //
174 // * `N`: the number of signals in the first `bus` (int, as a constant numerical expression)
175 // * `M`: the number of signals in the second `bus` (int, as a constant numerical expression)
176 //
177 // #### Test
178 // ```
179 // ro = library("routes.lib");
180 // os = library("oscillators.lib");

```



```

181 // crossNM_test = (os.osc(180), os.osc(280), os.osc(380), os.osc(480), os.osc(580)) : ro.
      crossNM(2,3);
182 // ```
183 //-----
184 crossNM(N,M) = route(N+M, N+M, par(i, N+M, i+1, ((i+M)%(N+M))+1));
185
186
187 //-----`(ro.)interleave`-----
188 // Interleave R x C cables from column order to row order. That is, transpose the input CxR
      matrix,
189 // the first R inputs is the first row.
190 //
191 // input : `x(0), x(1), x(2) ..., x(row*col-1)`
192 //
193 // output: `x(0+0*row), x(0+1*row), x(0+2*row), ..., x(1+0*row), x(1+1*row), x(1+2*row), ...`
194 //
195 //
196 //
197 // #### Usage
198 //
199 // ```
200 // si.bus(R*C) : interleave(R,C) : si.bus(R*C)
201 // ```
202 //
203 // Where:
204 //
205 // * `R`: row length (int, as a constant numerical expression)
206 // * `C`: column length (int, as a constant numerical expression)
207 //
208 // #### Test
209 // ```
210 // ro = library("routes.lib");
211 // os = library("oscillators.lib");
212 // interleave_test = (os.osc(200), os.osc(300), os.osc(400), os.osc(500)) : ro.interleave
      (2,2);
213 // ```
214 //-----
215 interleave(1,2) = _,-;
216 interleave(R,C) = route(R*C, R*C, par(i, R*C, (i+1, (i/R)*C + int(i/R) + 1)));
217
218 //-----`(ro.)butterfly`-----
219 // Addition (first half) then subtraction (second half) of interleaved signals.
220 //
221 // #### Usage
222 //
223 // ```
224 // si.bus(N) : butterfly(N) : si.bus(N)
225 // ```
226 //
227 // Where:
228 //
229 // * `N`: size of the butterfly (N is int, even and as a constant numerical expression)
230 //
231 // #### Test
232 // ```
233 // ro = library("routes.lib");
234 // os = library("oscillators.lib");
235 // butterfly_test = (os.osc(250), os.osc(350), os.osc(450), os.osc(550)) : ro.butterfly(4);
236 // ```
237 //-----
238 butterfly(2) = si.bus(2) <: +,-;
239 butterfly(N) = si.bus(N) <: interleave(N/2,2), interleave(N/2,2) : par(i, N/2, +), par(i, N
      /2, -);
240
241
242 //-----`(ro.)hadamard`-----
243 // Hadamard matrix function of size `N = 2^k`.
244 //

```

```

245 // #### Usage
246 //
247 // ```
248 // si.bus(N) : hadamard(N) : si.bus(N)
249 // ```
250 //
251 // Where:
252 //
253 // * `N`: `2^k`, size of the matrix (int, as a constant numerical expression)
254 //
255 // #### Test
256 // ```
257 // ro = library("routes.lib");
258 // os = library("oscillators.lib");
259 // hadamard_test = (os.osc(220), os.osc(330), os.osc(440), os.osc(550)) : ro.hadamard(4);
260 // ```
261 //-----
262 declare hadamard author "Remy Muller, revised by Romain Michon";
263 hadamard(2) = butterfly(2);
264 hadamard(N) = butterfly(N) : (hadamard(N/2), hadamard(N/2));
265
266 //-----`(ro.)recursiveize`-----
267 // Create a recursion from two arbitrary processors `p` and `q`.
268 //
269 // #### Usage
270 //
271 // ```
272 // _ : recursiveize(p,q) : _
273 // ```
274 //
275 // Where:
276 //
277 // * `p`: the forward arbitrary processor
278 // * `q`: the feedback arbitrary processor
279 //
280 // #### Test
281 // ```
282 // ro = library("routes.lib");
283 // os = library("oscillators.lib");
284 // recursiveize_test = (os.osc(220), os.osc(330)) : ro.recursiveize(*(0.5), *(0.3));
285 // ```
286 //-----
287 recursiveize(p,q) = (_,_,_ :> sp.stereoize(p)) ~ sp.stereoize(q);
288
289 //-----`(ro.)bubbleSort`-----
290 //
291 // Sort a set of N parallel signals in ascending order on-the-fly through
292 // the Bubble Sort algorithm.
293 //
294 // Mechanism: having a set of N parallel signals indexed from 0 to N - 1,
295 // compare the first pair of signals and swap them if sig[0] > sig[1];
296 // repeat the pair comparison for the signals sig[1] and sig[2], then again
297 // recursively until reaching the signals sig[N - 2] and sig[N - 1]; by the end,
298 // the largest element in the set will be placed last; repeat the process for
299 // the remaining N - 1 signals until there is a single pair left.
300 //
301 // Note that this implementation will always perform the worst-case
302 // computation, O(n^2).
303 //
304 // Even though the Bubble Sort algorithm is one of the least efficient ones,
305 // it is a useful example of how automatic sorting can be implemented at the
306 // signal level.
307 //
308 // #### Usage
309 //
310 //
311 //
312 //

```

```

313 // ```
314 // si.bus(N) : bubbleSort(N) : si.bus(N)
315 //
316 // ```
317 //
318 // Where:
319 //
320 // * `N`: the number of signals to be sorted (must be an int >= 0, as a constant numerical
    expression)
321 //
322 // #### Test
323 // ```
324 // ro = library("routes.lib");
325 // bubbleSort_test = (
326 //   hslider("bubbleSort:x0", 0.3, -1, 1, 0.01),
327 //   hslider("bubbleSort:x1", -0.2, -1, 1, 0.01),
328 //   hslider("bubbleSort:x2", 0.8, -1, 1, 0.01),
329 //   hslider("bubbleSort:x3", -0.5, -1, 1, 0.01)
330 // ) : ro.bubbleSort(4);
331 // ```
332 //
333 // #### References
334 //
335 // * <https://en.wikipedia.org/wiki/Bubble\_sort>
336 //-----
337 declare bubbleSort author "Dario Sanfilippo";
338 declare bubbleSort copyright "Copyright (C) 2021 Dario Sanfilippo
339   <sanfilippo.dario@gmail.com>";
340 declare bubbleSort license "MIT License";
341 bubbleSort(0) = 0 : !;
342 bubbleSort(1) = _;
343 bubbleSort(N) = seq(i, N - 1, pairSortN(N - i), bus(i))
344   with {
345     bus(0) = 0 : !;
346     bus(N) = si.bus(N);
347     pairSort = bus(2) <: select2(>), select2(<);
348     pairSortN(N) = seq(i, N - 1, bus(i), pairSort, bus(N - i - 2));
349   };
350
351
352 //------(ro.)bitonicSort-----
353 //
354 // A bitonic sorter is a parallel sorting network that performs comparisons
355 // and swaps on signal pairs iteratively. The pairs are selected according
356 // to different index gaps determined by the current state and pass indices,
357 // while the sorting direction of each pair depends on the current stage index.
358 //
359 // The algorithm first builds a bitonic sequence with the input values,
360 // that is, a sequence that is ascending for the first half, and descending
361 // for the second half. Then, it applies a bitonic merger to sort the bitonic
362 // list in ascending or descending order.
363 //
364 // The algorithm has  $O(\log^2(N) \cdot N)$  depth complexity, and  $O(N \cdot \log^2(N))$ 
365 // work complexity. Specifically, the bitonic sorter algorithm requires  $N$ 
366 // inputs to be a power-of-two. The algorithm has a total of
367 //  $\log^2(N) \cdot ((\log^2(N) + 1) / 2)$  stages, and each stage requires  $N / 2$ 
368 // comparators. Each comparator has one min() and one max() function.
369 //
370 // For comparison, the Bubble Sort algorithm has  $N - 1$  stages, and
371 //  $N \cdot (N - 1) / 2$  comparators. For  $N = 32$ , Bitonic Sort has 15 stages
372 // and 240 comparators, Bubble Sort has 31 stages and 496 comparators.
373 //
374 // The sorting process is zero-latency.
375 //
376 // #### Usage
377 //
378 // ```
379 // si.bus(N) : bitonicSort(N) : si.bus(N)

```

```

380 //
381 // ...
382 //
383 // Where:
384 //
385 // * `N`: the number of signals to be sorted (it must be a power-of-two
386 // compile-time numerical expression)
387 //
388 // #### Test
389 // ...
390 // ro = library("routes.lib");
391 // bitonicSort_test = (
392 //   hslider("bubbleSort:x0", 0.3, -1, 1, 0.01),
393 //   hslider("bubbleSort:x1", -0.2, -1, 1, 0.01),
394 //   hslider("bubbleSort:x2", 0.8, -1, 1, 0.01),
395 //   hslider("bubbleSort:x3", -0.5, -1, 1, 0.01)
396 // ) : ro.bitonicSort(4);
397 // ...
398 //
399 // #### References
400 //
401 // * <https://en.wikipedia.org/wiki/Bitonic\_sorter>
402 declare bitonicSort author "Dario Sanfilippo";
403 declare bitonicSort copyright "Copyright (C) 2026 Dario Sanfilippo
404   <sanfilippo.dario@gmail.com>";
405 declare bitonicSort license "MIT License";
406
407 /// Helper 0-indexed route wrapper
408 route0idx(N, M, connectionList) =
409   route(N, M, si.vecOp((connectionList, par(i, outputs(connectionList), 1)), +));
410 /// Ascending and descending comparators
411 comparatorDes(x0, x1) = max(x0, x1) , min(x0, x1);
412 comparatorAsc(x0, x1) = min(x0, x1) , max(x0, x1);
413 comparator(0, x0, x1) = comparatorDes(x0, x1);
414 comparator(1, x0, x1) = comparatorAsc(x0, x1);
415 /// Direction selector, according to the stage index
416 comparatorDirections(N, s) = par(i, N / 2, (i % (2 ^ (s + 1)) < (2 ^ s)));
417 /// Comparison input routing pairs
418 bitonicRouteCompareInputList(N, G) = inlets , outlets : interleave(N, 2) with {
419   inlets = par(j, N / (2 * G) , par(i, G, i + j * (2 * G) , i + G + j * (2 * G)));
420   outlets = par(i, N, i);
421 };
422 /// Comparison output unrouting pairs
423 bitonicRouteCompareOutputList(N, G) = inlets , outlets : interleave(N, 2) with {
424   inlets = par(i, N, i);
425   outlets = par(j, N / (2 * G) , par(i, G, i + j * (2 * G) , i + G + j * (2 * G)));
426 };
427 /// Apply routing
428 bitonicRouteInput(N, G) = route0idx(N, N, (bitonicRouteCompareInputList(N, G)));
429 bitonicRouteOutput(N, G) = route0idx(N, N, (bitonicRouteCompareOutputList(N, G)));
430 /// Bitonic sequence builder layer
431 bitonicBuilderLayer(N, s, p) = Y with {
432   G = 2 ^ (s - p);
433   D(i) = ba.pick(comparatorDirections(N, s), i);
434   Y = bitonicRouteInput(N, G) : par(i, N / 2, comparator(D(i))) : bitonicRouteOutput(N, G);
435 };
436 /// Bitonic sequence builder network
437 bitonicBuilderNetwork(N) = Y with {
438   S = rint(log2(N) - 1);
439   Y = seq(s, S, seq(p, s + 1, bitonicBuilderLayer(N, s, p)));
440   log2(x) = log(x)/log(2.0);
441 };
442 /// Bitonic sorter layer
443 bitonicSorterLayer(N, s, p) = Y with {
444   G = 2 ^ (s - p);
445   Y = bitonicRouteInput(N, G) : par(i, N / 2, comparator(1)) : bitonicRouteOutput(N, G);
446 };
447 /// Bitonic sorter network

```

```

448 bitonicSorterNetwork(N) = Y with {
449   S = rint(log2(N) - 1);
450   Y = seq(p, S + 1, bitonicSorterLayer(N, S, p));
451   log2(x) = log(x)/log(2.0);
452 };
453 /// Bitonic sorter
454 bitonicSort(N) = bitonicBuilderNetwork(N) : bitonicSorterNetwork(N);
455
456 ///------(ro.)bitonicSortIdx-----
457 ///
458 /// This function is based on the `bitonicSort` function, except it returns the
459 /// set of indices for the ordered input values. This is useful if you want
460 /// to sort one set based on another one.
461 ///
462 /// #### Usage
463 ///
464 /// ...
465 /// si.bus(N) : bitonicSortIdx(N) : si.bus(N)
466 ///
467 /// ...
468 ///
469 /// Where:
470 ///
471 /// * `N`: the number of signals to be sorted (it must be a power-of-two
472 /// compile-time numerical expression)
473 ///
474 /// #### Test
475 /// ...
476 ///
477 /// ro = library("routes.lib");
478 /// bitonicSortIdx_test = (
479 ///   hslider("bubbleSort:x0", 0.3, -1, 1, 0.01),
480 ///   hslider("bubbleSort:x1", -0.2, -1, 1, 0.01),
481 ///   hslider("bubbleSort:x2", 0.8, -1, 1, 0.01),
482 ///   hslider("bubbleSort:x3", -0.5, -1, 1, 0.01)
483 /// ) : ro.bitonicSortIdx(4);
484 /// ...
485 ///
486 /// #### References
487 ///
488 /// * <https://en.wikipedia.org/wiki/Bitonic\_sorter>
489 declare bitonicSortIdx author "Dario Sanfilippo";
490 declare bitonicSortIdx copyright "Copyright (C) 2026 Dario Sanfilippo
491   <sanfilippo.dario@gmail.com>";
492 declare bitonicSortIdx license "MIT License";
493
494 comparatorDesIdx(x0, i0, x1, i1) = ba.if(cond, x1, x0) , ba.if(cond, i1, i0) , ba.if(cond,
495   x0, x1) , ba.if(cond, i0, i1) with {
496   cond = x1 > x0;
497 };
498 comparatorAscIdx(x0, i0, x1, i1) = ba.if(cond, x1, x0) , ba.if(cond, i1, i0) , ba.if(cond,
499   x0, x1) , ba.if(cond, i0, i1) with {
500   cond = x0 > x1;
501 };
502 comparatorIdx(0, x0, i0, x1, i1) = comparatorDesIdx(x0, i0, x1, i1);
503 comparatorIdx(1, x0, i0, x1, i1) = comparatorAscIdx(x0, i0, x1, i1);
504 comparatorDirectionsIdx(N, s) = par(i, N / 2, (i % (2 ^ (s + 1)) < (2 ^ s)));
505
506 bitonicRouteCompareInputListIdx(N, G) = inlets , outlets : interleave(2 * N, 2) with {
507   inlets = par(j, N / (2 * G) , par(i, G,
508     2 * (i + j * (2 * G)) ,
509     2 * (i + j * (2 * G)) + 1 ,
510     2 * (i + G + j * (2 * G)) ,
511     2 * (i + G + j * (2 * G)) + 1
512   ));
513   outlets = par(i, N, 2 * i , 2 * i + 1);
514 };
515 bitonicRouteCompareOutputListIdx(N, G) = inlets , outlets : interleave(2 * N, 2) with {

```

```

514   inlets = par(i, N, 2 * i, 2 * i + 1);
515   outlets = par(j, N / (2 * G), par(i, G,
516     2 * (i + j * (2 * G)),
517     2 * (i + j * (2 * G)) + 1,
518     2 * (i + G + j * (2 * G)),
519     2 * (i + G + j * (2 * G)) + 1
520   ));
521 };
522 bitonicRouteInputIdx(N, G) = route0idx(2 * N, 2 * N, (bitonicRouteCompareInputListIdx(N, G))
523   );
524 bitonicRouteOutputIdx(N, G) = route0idx(2 * N, 2 * N, (bitonicRouteCompareOutputListIdx(N, G
525   )));
526
527 bitonicBuilderLayerIdx(N, s, p) = Y with {
528   G = 2 ^ (s - p);
529   D(i) = ba.pick(comparatorDirectionsIdx(N, s), i);
530   Y = bitonicRouteInputIdx(N, G) : par(i, N / 2, comparatorIdx(D(i))) :
531     bitonicRouteOutputIdx(N, G);
532 };
533 bitonicBuilderNetworkIdx(N) = Y with {
534   S = rint(log2(N) - 1);
535   Y = seq(s, S, seq(p, s + 1, bitonicBuilderLayerIdx(N, s, p)));
536   log2(x) = log(x)/log(2.0);
537 };
538 bitonicSorterLayerIdx(N, s, p) = Y with {
539   G = 2 ^ (s - p);
540   Y = bitonicRouteInputIdx(N, G) : par(i, N / 2, comparatorIdx(1)) : bitonicRouteOutputIdx(
541     N, G);
542 };
543 bitonicSorterNetworkIdx(N) = Y with {
544   S = rint(log2(N) - 1);
545   Y = seq(p, S + 1, bitonicSorterLayerIdx(N, S, p));
546   log2(x) = log(x)/log(2.0);
547 };
548 bitonicSortIdx(N) = si.bus(N), par(i, N, i) : interleave(N, 2) : bitonicBuilderNetworkIdx(N
549   ) : bitonicSorterNetworkIdx(N) : par(i, N, !, _);

```